

Mikko Kuusisto

PARAMETER OPTIMIZATION FOR RETRIEVAL-AUGMENTED GENERATION SYSTEMS

Masters Thesis
Faculty of Engineering and Natural Sciences
Examiners: Prof. Jaakko Peltonen
Dr. Simo Ali-Löytty
December 2025

ABSTRACT

Mikko Kuusisto: Parameter Optimization for Retrieval-Augmented Generation Systems
Masters Thesis
Tampere University
Master's Programme in Science and Engineering
December 2025

Retrieval-Augmented Generation (RAG) is a modeling approach that incorporates domain-specific context into Large Language Models (LLMs) in order to provide more accurate answers to user queries. This thesis investigates how answers generated by RAG models can be effectively evaluated using suitable metrics and identifies optimal context-related parameters for such models. The large language models used in this work are based on the transformer architecture, which predicts the most probable tokens using attention mechanisms. Retrieval-Augmented Generation employs hybrid search, combining semantic search with traditional keyword-based search to retrieve the most relevant segments of the context, referred to as chunks. Using the retrieved context, predefined instructions, and the user query, the RAG model generates a response.

The generated answers are evaluated using the RAGAS library, which provides multiple metrics for RAG validation. In this thesis, answer similarity, answer correctness, context precision, and context recall are used to assess overall accuracy and contextual relevance. Some of these metrics require a ground truth answer, which serves as a reference for comparison with the model-generated response. The ground truth is constructed by a domain specialist to ensure correctness and contextual appropriateness. Each metric evaluates a different aspect of the RAG pipeline, and together they provide a reliable validation framework.

Bayesian optimization techniques are employed to optimize the context parameters, specifically the number of retrieved chunks (Top-N) and the chunk size. Bayesian optimization is well suited for optimizing black-box functions with high evaluation costs, making it an appropriate choice given the computational expense of answer evaluation. In the test setup, the Top-N parameter is limited to the range 1–10, and the chunk size is selected from the set {500, 2000, 5000, 10000, 15000}.

The optimization results indicate that the best-performing values for Top-N lie between 6 and 8, with the strongest performance observed at 6, while 7 and 8 yield comparable results. The optimal chunk size is found to be 2000 tokens. The results indicate that very low or very high values of the Top-N parameter, as well as extremely small or large chunk sizes, lead to a decrease in the overall evaluation metric. In this thesis, the evaluation metrics are unweighted, which results in context precision and context recall having a greater influence on the overall score.

Future work may explore weighting the evaluation metrics according to the specific objectives of the RAG model. Additionally, the number of evaluation questions can be increased, and the evaluation methodology can be further refined to strengthen the validation of the results presented in this study.

Keywords: AI, Artificial Intelligence, Bayesian Optimization, Evaluation, Parameter Optimization, RAG, Retrieval Augmented Generation

The originality of this thesis has been checked using the Turnitin OriginalityCheck service.

TIIVISTELMÄ

Mikko Kuusisto: Hakupohjaisten generointimallien parametrien optimointi
Diplomityö
Tampereen yliopisto
Teknis-luonnontieteellinen
Joulukuu 2025

Hakupohjainen generointimalli (eng. RAG) on suuriin kielimalleihin perustuva malli, jonka avulla voidaan vastata tarkasti alakohtaisiin kysymyksiin. RAG malli hyödyntää valittuja dokumentteja ja kontekstina, josta suuri kielimalli hakee vastaukset kysymyksiin. Tässä työssä tutkitaan, miten RAG-mallien tuottamia vastauksia voidaan arvioida tehokkaasti ja millä kontekstiin liittyvillä parametreilla malli antaa parhaita vastauksia. Diplomityössä käytetyn suuren kielimallin arkkitehtuuri perustuu transformer-arkkitehtuuriin, joka ennustaa todennäköisimpää tekstin alkia (eng. token). Transformer-arkkitehtuuri muodostaa tekstiä syöttämällä todennäköisimmän tekstin alkion huomiomekanismin (eng. Attention) avulla. RAG-malli käyttää hybridihaun menetelmää, joka yhdistää semanttisen ja avainsanahaun, jotta kontekstista voidaan löytää oleelliset kappaleet (eng. chunks). RAG-malli tuottaa vastauksen käyttäen kontekstia, ennalta määritettyjä ohjeita ja käyttäjän kyselyä.

Tuotettuja vastauksia arvioidaan RAGAS-kirjaston avulla, joka tarjoaa useita mittareita RAG-mallien tuottamien vastausten arviointiin. Tässä työssä käytetään mittareina vastausten samankaltaisuutta, oikeellisuutta, kontekstin tarkkuutta (eng. precision) ja kontekstin kattavuutta (eng. recall) arvioitaessa vastauksien tarkkuutta ja sitä, että annettu vastaus on kontekstuaalisesti tarkka. Osa mittareista vaatii ihmisen antaman vastauksen (eng. Ground Truth), johon mallin tuottamia vastauksia verrataan. Kyseiset vastauksen laatii alan asiantuntija varmistaakseen niiden oikeellisuuden. Kukin mittari arvioi RAG-prosessin eri osa-aluetta, ja yhdessä ne muodostavat luotettavan arviointikehyksen.

Kontekstiparametrien optimointiin käytetään Bayesilaista optimointia. Optimoitavia parametreja työssä ovat haettavien kappaleiden määrä (Top-N) ja kappaleiden koko (Chunk Size). Bayesilainen optimointi soveltuu hyvin määrittelemättömien funktioiden optimointiin, joiden arviointi on kustannuksiltaan kallista. Tässä työssä käytettävät evaluointimetriikat ovat laskennallisesti vaativia. Tästä syystä Bayesilainen optimointi soveltuu työn ongelman ratkaisemiseen. Testiasetuksissa Top-N-parametri on rajoitettu välille 1–10, ja chunkin koko valitaan joukosta {500, 2000, 5000, 10000, 15000}.

Optimointitulokset osoittavat, että parhaat Top-N-arvot sijoittuvat välille 6–8, ja paras suorituskyky havaittiin arvolla 6, kun taas arvot 7 ja 8 tuottivat lähes vastaavat tulokset. Optimaalinen chunkin koko on 2000 tokenia. Tuloksista havaitaan, että hyvin alhaiset tai hyvin korkeat Top-N-arvot, samoin kuin äärimmäisen pienet tai suuret chunkien koot, johtavat tulosten heikkenemiseen. Tässä työssä arviointimittarit on jätetty painottamatta. Työssä havaittiin kontekstin tarkkuuden (precision) ja kattavuuden (recall) vaikuttavat muita mittareita enemmän optimointituloksiin.

Tulevaisuudessa arviointimittareiden painotusta voidaan muokata RAG-mallin tavoitteiden mukaisesti. Lisäksi arvioitavien kysymysten määrää voidaan lisätä ja arviointimenetelmää edelleen kehittää tulosten validoinnin vahvistamiseksi.

Avainsanat: AI, arviointi, bayesian optimointi, hakupohjainen generointimalli, optimointi, tekoäly

Tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck -ohjelmalla.

USE OF AI IN THESIS

I have utilised AI tools in my thesis:

☐ No

☒ Yes

The AI tools utilised in my thesis and their purposes are described below:

Names and versions of AI tools: GPT-4o, text-embedding-ada-002

Purpose of using AI tools:

- Improving clarity and readability of text.
- Spell checking and language refinement.
- Supporting in debugging Python for my optimizer.
- Generating LaTeX for tables and mathematical notations.
- Generating answers and evaluation metrics for Retrieval-Augmented Generation (RAG) experiments.

Sections where AI tools were used:

- Preface and main text for language clarity and LaTeX code.
- Results chapter for metric generation and Python support.

I acknowledge that I am fully responsible for the entire content of my thesis, including the parts generated by AI, and accept accountability for any violations of ethical standards in publications.

PREFACE

First of all, I want to thank my employer, Epical, for providing the thesis topic and supporting me throughout the process. I could not have evaluated the answers given by the large language model without my dear friend, LL.M. Markus Tolvanen, who constructed the ground truth answers using his deep knowledge of insurance law and his impressive patience. I also want to thank my friends and family, who made fun of my never-ending thesis journey so relentlessly that I was eventually forced to finish it. For the rock-solid home supporters, Ella and Vilda, I want to express my deepest gratitude.

It's been a while since I last identified myself as a student. Looking back at the years spent in vague basements, lecture halls, workshops, and at home with my dear ex-roommate Miska, I genuinely can't think of a single bad moment. I am deeply grateful to the Finnish education system for funding my studies. Without you, this would not exist, my old friend hyvinvointiyhteiskunta. I also want to give a shout-out to NMKSV for all the support I received during my studies and throughout this process. And yes, I believe I promised Pablo I would thank him in my preface, purely for the good times.

Finally, to all the fellow scientists out there studying how artificial intelligence will take our jobs and make us pay for the rude comments we once made about ChatGPT: until I see an AI pull an all-nighter, ace an exam the next morning, and still be at least ten beers deep, keep on optimizing <3

In Helsinki, 31st December 2025

Mikko Kuusisto

CONTENTS

1. Introduction	1
2. Generative Artificial Intelligence	4
2.1 Natural Language Processing	4
2.2 Classification	6
2.3 Embedding Model	7
2.4 Neural Networks	8
2.5 Large Language Models	8
2.6 Transformers	9
3. Retrieval Augmented Generation	13
3.1 Chunking	14
3.2 Context Retrieval	16
3.3 Answer Generation	19
4. Evaluating a RAG Application	21
4.1 Evaluation Framework	21
4.2 Statistical Analysis	22
4.3 Retrieval Evaluation	24
4.4 Generation Evaluation	26
5. Bayesian Optimization	28
6. Optimizing RAG Parameters	34
6.1 Related Work	34
6.2 Test Setup	35
6.3 Optimization	37
7. Results	38
8. Conclusions	44
References	46
Appendix A: RAG Context Documents	51
Appendix B: Test Setup Parameters	52

LIST OF SYMBOLS AND ABBREVIATIONS

A	Answer vector
C	Confusion matrix
D	Document
F	F-measure
FN	False Negative
FP	False Positive
G	Ground truth vector
K	Key vector for tokens compared against the query vector
P	Precision
Q	Current token in transformer
R	Recall
TN	True Negative
TP	True Positive
V	Value vector of the output after attention mechanism
W	Projection matrix
Θ	Weight matrix
α	attention weight for token
β	Relative importance factor
$\mathcal{C}$	Retrieved Context
$\mathcal{D}$	Dataset
$\mathcal{GP}$	Gaussian Process
$\mathcal{G}$	Ground truth set of claims
b	Bias term
c	claim
d_k	Dimension of the key value
f	Feature
h	Output of the token-wise feed-forward sublayer

k -NN	k -Nearest Neighbors
q	Query
$r(d)$	Position of the document in the list
v_k	Relevancy indicator
z	Output vector of transformer
AI	Artificial Intelligence
ANN	Approximate Nearest Neighbor
EI	Expected Improvement
FNN	Feed Forward Network
Gen AI	Generative Artificial Intelligence
GP	Gaussian Process
GPT	Generative Pre-trained Transformer
GT	Ground Truth
HNSW	Hierarchical Navigable Small World
IDF	Inverse Document Frequency
LLM	Large Language Model
NLP	Natural Language Processing
P	Precision
R	Recall
RAG	Retrieval Augmented Method
ReLu	Rectified Linear Unit
RNN	Recurrent Neural Network
RRF	Reciprocal Rank Fusion

1. INTRODUCTION

In recent years, Artificial Intelligence (AI) has advanced rapidly and become an integral component of many technical and industrial workflows. Rather than serving as a distant or speculative technology, AI now plays a practical role in supporting decision-making, automating processes, and enabling new forms of data-driven analysis. This thesis examines a specific branch of generative AI, Retrieval-Augmented Generation (RAG), with the aim of evaluating its performance and identifying parameter configurations that enhance its effectiveness.

Generative AI, particularly Generative Pre-trained Transformers (GPT) and other large language models (LLMs), has seen rapid advancements. These models are trained on vast amounts of data, encompassing a wide range of topics. However, the general nature of this data raises a challenge for companies seeking to apply these models to specific contexts. For instance, when asking a generative AI about workplace procedures, it may provide a generic response rather than referring to specific company policies.

To address this issue, generative AI can be enhanced with a database of specific documents, enabling it to retrieve relevant information in particular situations. This method, known as Retrieval Augmented Generation, allows companies to leverage their own data within large language models. Although the example of workplace procedures may seem simple, the RAG method offers numerous opportunities to address various challenges associated with LLMs, such as output inaccuracies and hallucinations.

The rapid advancement of large language models has prompted a critical question: Is retrieval-augmented generation the most effective method for generating context-specific responses? In recent years, the context window size that LLMs can process has increased by an order of magnitude, significantly expanding their capacity to handle larger documents. Should the RAG system have as much context as possible? If not, how to evaluate and optimize the system to have optimal parameters for generating accurate and reliable answers.

Having said this, the research question for this thesis is what metrics can be employed to effectively evaluate RAG applications and what are the optimal context parameters that maximize their performance? The primary goal of this study is to identify the most effective context-related parameters for RAG systems. By systematically tuning these

parameters, we aim to enhance the performance of RAG applications in generating accurate and contextually relevant responses. In addition to parameter optimization, this thesis seeks to study evaluation metrics that can reliably measure the quality and efficiency of RAG systems. Establishing robust metrics is essential for guiding optimization processes and ensuring that improvements are meaningful and reproducible.

This thesis investigates the limitations and potential of the RAG approach in relation to context size. We aim to optimize the performance of RAG-based systems by tuning context-related parameters using general-purpose optimization techniques that are applicable across a wide range of problem domains. The overview of the study is the use of Bayesian optimization methodologies to enhance RAG performance. In a nutshell, the whole process is to use the Bayesian optimizer to select parameters for the RAG application that produces answers for certain set of queries. The given answers are evaluated, and the evaluation scores are passed back to the optimizer. Here is a simple graph representing the optimization workflow.

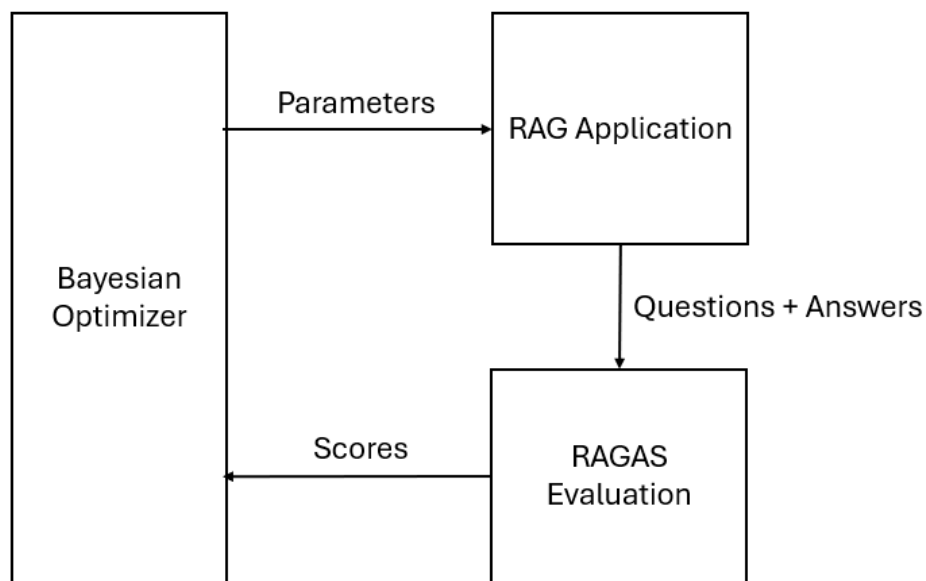


Figure 1.1. Optimization flow overview.

To effectively optimize the application, it is essential to establish a framework for evaluating the quality of generated responses. In this work, we propose an evaluation framework for assessing the performance of AI applications. The evaluation results are then used to guide the optimization of context parameters.

The objective of this thesis extends beyond context optimization. The evaluation methodologies introduced are designed to ensure that generative models produce accurate and reliable outputs. By systematically evaluating the RAG method, we aim to enhance the overall reliability and effectiveness of generative AI systems.

Chapter 2 sets the foundation by introducing the core concepts of generative artificial intelligence (GenAI), providing the necessary background to comprehend how these models function and generate outputs. Building on this foundation, Chapter 3 explores the retrieval-augmented generation (RAG) framework. We examine its architecture, discuss how it can be adapted for different use cases, and introduce a hybrid search strategy designed to improve the retrieval of relevant context to generate accurate responses.

With a solid understanding of RAG systems, Chapter 4 introduces an evaluation framework. This chapter outlines the metrics used to assess the quality and reliability of generated responses, which in turn guide the optimization process. Chapter 5 shifts focus to optimization techniques. We provide an overview of Bayesian optimization and demonstrate how it can be applied to fine-tune parameters, such as context, size to enhance system performance.

The Chapter 6 introduces the test setup used in the thesis. The chapter 7 presents the experimental results and chapter 8 offers a detailed discussion of the findings. These chapters evaluate the effectiveness of the proposed methods and reflect on their implications for the development of more accurate and reliable generative AI applications.

2. GENERATIVE ARTIFICIAL INTELLIGENCE

Generative AI refers to artificial intelligence techniques that generate new content such as text, images, music, and more by learning from large amounts of data. [57].

The development of next-generation AI applications is progressing rapidly, with a growing demand for personalized AI solutions. It is crucial to design models that provide answers that meet the required specifications. Large language models are usually not the application itself. Evaluating whether different large language models improve the outcome of the application is a difficult task. The overall performance depends on the application and the impact of the language model can be minimal [21].

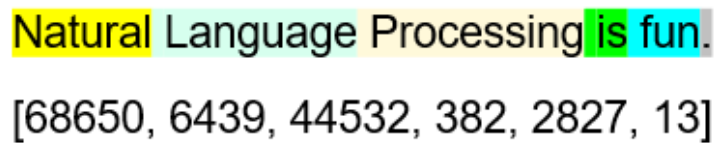
This thesis will explore methods for evaluating generative AI applications and assess how modifications to the model impact the responses it generates. This work proposes a framework for evaluating the responses generated by AI applications and examines how this evaluation affects the deployment process. Furthermore, the behavior of generative AI models can vary in different contexts, making it essential to evaluate applications designed for specific purposes. The work also proposes a method for optimizing the parameters of an AI application. Once an application can be evaluated, it enables us to optimize the parameters for even better results. In the chapter 6, we are concentrating on parameter optimization and how RAG applications can be modified.

In this section, we will also dive into the underlying algorithms of GPT models to clarify the operational logic of AI applications.

2.1 Natural Language Processing

To generate answers to user's questions, the so-called prompts, large language models must be able to process the underlying meaning of the input. Human communication relies on complex language structures with varied semantic nuances. Natural language processing has been developed to process the meaning of text, enabling LLMs to answer questions or generate relevant content [31]. Unlike humans, large language models do not comprehend text semantically, but rather find hidden patterns through algorithms described later in this chapter.

In natural language processing, dividing a text into smaller units is essential to understand its meaning. This segmentation aims to assign significance to individual parts, enabling subsequent analysis. This fundamental operation is known as tokenization. It involves dividing a text or character sequence into smaller units known as tokens. In NLP, these tokens usually represent words or sub-words and serve as fundamental components for language processing tasks. [2]. The word level tokenization of the phrase "Natural language processing is fun." would be as follows:



Natural Language Processing is fun.
[68650, 6439, 44532, 382, 2827, 13]

Figure 2.1. Tokenization example using OpenAI Tokenizer for GPT-4o. Figure adapted from [40].

Tokens are the fundamental building blocks for natural language processing. Before training, artificial intelligence models do not have understanding of words or sentences. To train a model to understand written text, tokens need to be classified according to their semantic meanings.

2.2 Classification

Classification is a process where an unknown data point x is assigned to a specific class label C_i from a predefined set C , where $C_i \in C$ [28]. Classification algorithms assist in organizing data into distinct classes, much like how books are categorized in a library based on various topics. Basic example of an classification task would be assign emails to spam folder based on the label "spam" or "non-spam". The label is defined using features of the email.

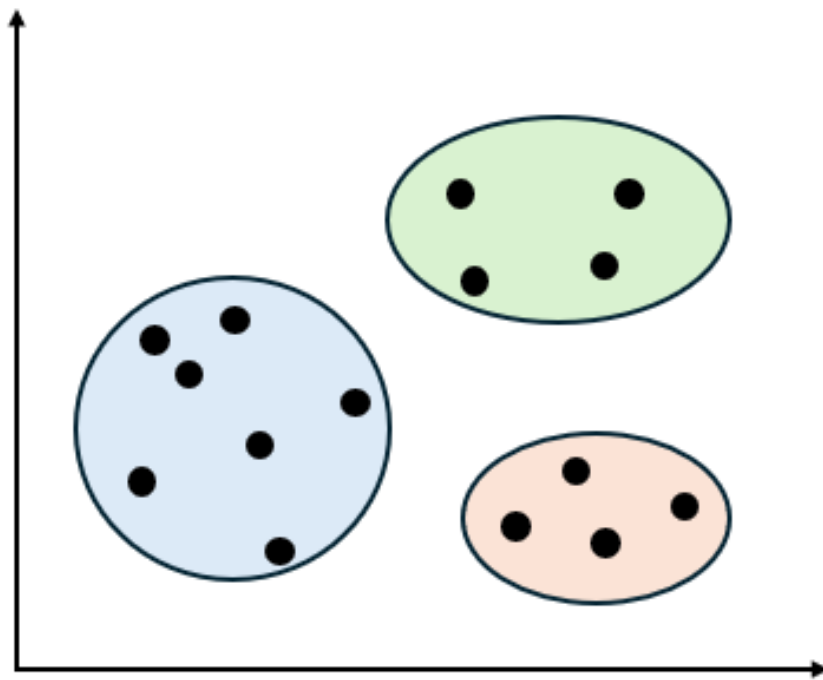


Figure 2.2. Data points classified into 3 categories based on their features.

In large language models, words are represented using their semantic meaning. However, instead of discrete separate classes, the semantic meaning is represented as vectors in high dimensional space where similar vectors represent similar meaning. Vector representation is called an embedding. The tokens are constructed as an embedding during the training phase of the llm. In the upcoming chapters, the principles of how the models capture the meaning are introduced. In chapter 3.2, we will introduce the cosine similarity that defines how similar two vectors are by looking at the angle between them. It evaluates how similarly tokens are represented in high dimensional space.

2.3 Embedding Model

The heart of an artificial intelligence application is the embedding model that creates vectors with semantic meaning out of natural language. In Natural Language Processing, text is commonly defined as a collection of words, sentences, or larger expression of meaningful information [16]. Embedding is a numerical vector in a certain space. Semantically similar embeddings are close to each other in the space. The context in which a word appears influences its meaning, and word embeddings are typically learned from this context. [10]

Text embeddings in natural language processing have undergone remarkable transformations during the last few decades [10]. One of the first vector representations of words was the bag of words. It represents words as a vector $x = [x_1, x_2, \dots, x_n]^T$, where x_j is the count of word j . Classification in decisions in linear classification tasks is done by a weighted total of individual feature counts, like word counts and the vector x is called a bag of words. The approach is only taking into account the count of words. It ignores semantic meaning, grammar, and everything else than words.[21]

To predict a label from a bag-of-words, each word in the vocabulary is assigned a score, or weight, indicating its compatibility with the label. These weights are organized in a column vector θ [21]. It is called also a word embedding.

Advancements in text representations were marked by the development of static dense word embeddings. Models like Word2Vec [36] and GloVe [44] generated dense vector representations based on the surroundings of words. Word2Vec utilized local context, GloVe incorporated global corpus statistics. Despite their ability to capture semantic and syntactic similarities, these models provided static vectors, which ignored the context-dependent meanings of words.[10]

The introduction of contextualized embeddings ushered in a new era of dynamic embeddings that adapt based on context. Notable models in this category include Embeddings from Generative Pre-trained Transformer and Bidirectional Encoder Representations from Transformers (BERT). GPT utilized transformers with unsupervised pre-training, and BERT leveraged bidirectional transformers for deeper contextual understanding.[10]

The latest developments in text embeddings focus on creating universal models that can support a wide range of downstream tasks. Advances in diverse datasets, synthetic data, and benchmarks such as the Massive Text Embedding Benchmark (MTEB) have improved their generalization across tasks and domains [10]. In this work, we are using an universal text embedding model from OpenAI, text-embedding-002-ada. The Embedding model uses transformer based approach to create embedding vectors from tokens. The tokens of sentences are then pooled as a single embedding vector. OpenAI has not published the exact internal pooling method used in the model.

2.4 Neural Networks

Among the most used tools, feedforward neural networks (FNN) constitute the basic functional units for a variety of Machine Learning implementations, covering hierarchical levels of complexity [23]. There is a comprehensive introduction to FNNs given here [21]. In this thesis, we will focus on transformer based models that are utilizing feedforward neural networks.

Recurrent Neural Networks (RNNs) are sequence-modeling architectures that are designed to identify patterns in sequential data, such as text, handwriting, genomic sequences, and industrial time series. RNNs can also be applied to images by breaking them down into patches and treating them as sequences. They are widely used in tasks like text generation, language modeling, image description generation, and speech recognition. [54]

The key difference between RNNs and Feedforward Neural Networks (also known as Multi-Layer Perceptrons or MLPs) lies in how they process information. Feedforward Networks pass information through the network without cycles, whereas RNNs contain cyclic connections that allow information to be fed back into the network itself. Through their recurrent design, RNNs can retain information from earlier inputs alongside current data, which strengthens their ability to handle sequential data. While the concept of multiple hidden layers is simplified into a single Hidden Layer block in illustrations, RNNs can indeed have multiple hidden layers. [54]

2.5 Large Language Models

Large Language Models form the foundation of modern conversational systems like ChatGPT. These models are trained on large datasets to recognize patterns and structures within the data, enabling them to produce coherent and contextually appropriate content. In Generative AI, LLMs focus on generating human-like text by predicting the next most likely word, and they are utilized for various natural language processing tasks, such as text completion, language translation, summarization, and more. [57]

Large language models are computational architectures that support language generation and other natural language processing tasks, including text classification. [37]. These models learn statistical relationships from large amounts of text data during a computationally exhaustive process of self-supervised and semi-supervised learning [45]. In this work, we are concentrating on generative artificial intelligence, that is a powered by large language models. Generative AI is capable to generate text, images, videos, or other data using pre-trained neural networks.

In 2017, Google published a paper titled “Attention is all you need”, which introduced the Transformer architecture that is an efficient neural network model for natural language processing tasks [58]. Unlike traditional models, the Transformer depends solely on attention mechanisms to process input sequences. By discarding the lengthy sequential steps present in other architectures and focusing only on attention mechanisms, this approach significantly improved the speed and parallel processing ability of the computation [17].

2.6 Transformers

For a long time, neural networks faced challenges related to computation time. Training neural networks using extensive datasets demands significant computing resources. The inherent sequential structure of RNNs prevents parallelization within training examples [43]. This limitation becomes particularly significant for long input sequences, as memory constraints restrict the ability to batch multiple examples. Even though there have been advancements in computational efficiency have been achieved through factorization techniques, the sequential computing constraint remains. [58]

In the paper ‘Attention is all you need’, the model architecture introduced departs from recurrence. Instead, it exclusively uses an attention mechanism to capture global dependencies between inputs and outputs.

There are two sides in the transformer architecture - encoder and decoder. In both, there are positional encoders, that provide information about the position of each token in the sentence. This is an essential part of the transformer, since it does not process words in order. The encoder uses multiple layers to process tokens multiple times to capture the semantic meaning efficiently. The encoder has multiple identical layers and all of them have two sub-layers. The first is a multi-head self-attention layer and the second is a feed-forward network. The output of each layer is used as the input of the next layer. The decoder also has multiple layers, but in addition to the two sub-layers, the decoder has a third sub-layer. It applies multi-head attention to the encoder stack’s output. As in the encoder, each sub-layer is equipped with a residual connection and subsequent layer normalization. In the decoder stack, we adjust the self-attention sub-layer to prevent any position from attending to future positions. By applying this masking technique and shifting the embeddings of the output by one position, the model makes sure that the prediction at position i depends only on the known outputs from the positions preceding i . [58]

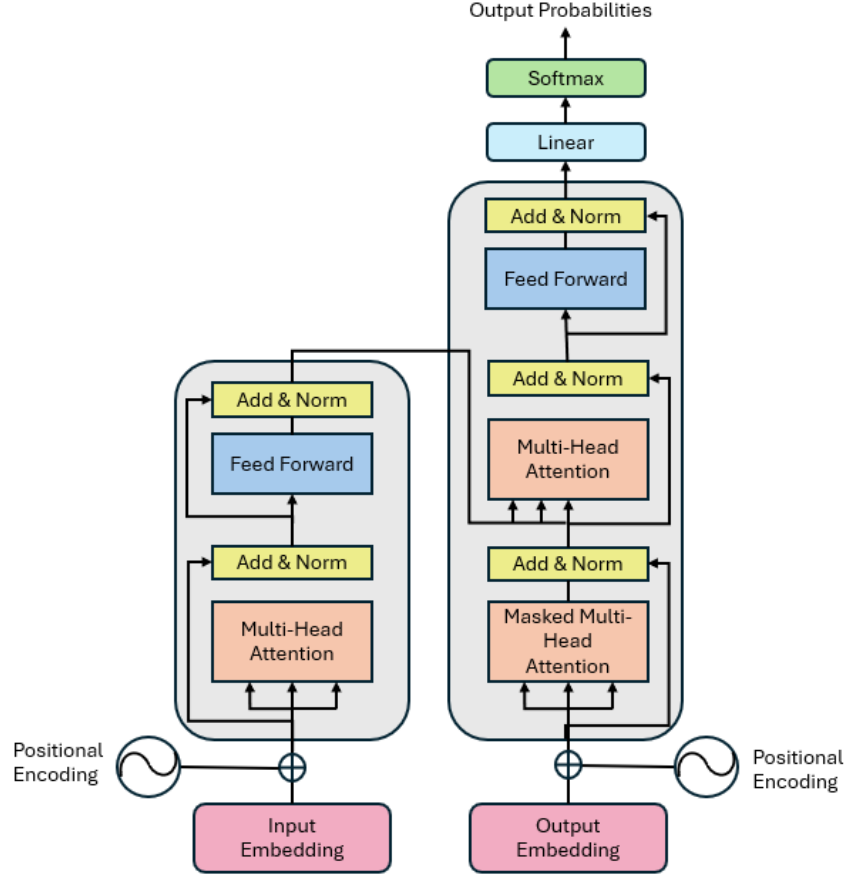


Figure 2.3. Transformer model architecture. Figure adapted from [58].

Attention mechanisms are a fundamental component of transformer models, enabling them to concentrate on different parts of the input sequence when making predictions. The two most frequently employed attention mechanisms are dot-product attention and additive attention.

Additive attention, introduced by Bahdanau et al. [3], determines alignment scores using a feed-forward neural network consisting of a single hidden layer. This mechanism is particularly effective when the number of dimensions of the key values (d_k) increases, as it avoids the issue of vanishing gradients in the softmax function [8].

Dot-product attention is computationally more efficient and space-saving compared to additive attention. However, it can suffer from small gradient values when d_k is large [58]. To mitigate this, a scaling factor of $\sqrt{d_k}$ is introduced. The attention function for a group of queries is calculated as follows [58]:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.1)$$

Here, Q , K , and V represent the matrices of queries, keys, and values, respectively. The query vector Q represents the current token for which the model is trying to find

relevant information from other tokens in the sequence. The key vector K is the tokens in the sequence that the model is comparing against the query vector to find relevant information [58]. Softmax function is defined as

$$\text{softmax}(X)_{ij} = \frac{\exp(X_{ij})}{\sum_t \exp(X_{it})} [6], \quad (2.2)$$

where each output is between $[0,1]$ and all outputs sum to one. It is used in the attention mechanism because it converts similarity scores into a probability distribution that tells the model how much attention to pay to each token. Because of the exponential behavior, output increases a lot with a slightly larger similarity score. This helps the model to focus on the most similar tokens.

The attention scores $\alpha_{m \rightarrow n}^{(i)}$ are computed using the scaled dot-product attention mechanism and can be denoted as:

$$\alpha_{m \rightarrow n}^{(i)} = \text{softmax} \left(\frac{Q_m K_n^T}{\sqrt{d_k}} \right) \quad (2.3)$$

where Q_m is the query vector for token m , K_n is the key vector for token n , and d_k is the dimension of the key vectors.

Multi-head attention extends the concept of single attention by applying multiple attention functions, or *heads*, in parallel. Each head operates on different learned projections of the queries, keys, and values, enabling the model to capture diverse relationships within the input sequence. The outputs of all heads are concatenated and projected to form the final representation [58]:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad (2.4)$$

where each head i is defined as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (2.5)$$

with W_i^Q, W_i^K, W_i^V denoting the projection matrices for queries, keys, and values, and W^O the output projection matrix.

The attention weights are used to compute a weighted sum of the value vectors V_n , which are derived from the embeddings of all tokens through another learned weight matrix W^V . This weighted sum forms the output vector $z_m^{(i)}$ for the current token m . The self-attention

output for token m at layer i is given by [21]:

$$z_m^{(i)} = \sum_{n=1}^{M^{(s)}} \alpha_{m \rightarrow n}^{(i)} (\Theta_v h_n^{(i-1)}), \quad (2.6)$$

where $\alpha_{m \rightarrow n}^{(i)}$ is the attention weight from token m to token n , $h_n^{(i-1)}$ is the previous layer's representation, and Θ_v projects values.

After the self-attention mechanism, each token's representation is passed through a position-wise feed-forward network. The FFN contributes significantly to the model's ability to transform and enrich token-level features, even when attention is fixed or limited [7]. This component is applied independently to each token and consists of two linear transformations with a non-linear activation function in between [58][21]:

$$h_m^{(i)} = \Theta_2 \text{ReLU}(\Theta_1 z_m^{(i)} + b_1) + b_2, \quad (2.7)$$

where b_1 and b_2 are the bias terms of the first and second layers, respectively. ReLU is defined as $f(x) = \max(x, 0)$ [46]. Followed by residual connections and layer normalization[58]:

$$\text{output} = \text{LayerNorm}(x + \text{SelfAttention}(x)). \quad (2.8)$$

Transformers constitute the foundational architecture of modern large language models. Their design leverages self-attention to model global dependencies across tokens, enabling contextual understanding beyond local neighborhoods. The model uses self-attention to compute contextual embeddings for each token, meaning every word is represented in relation to all others in the input. This ensures nuanced understanding of meaning, syntax, and semantics. In autoregressive LLMs, the ultimate objective is to predict the next token given the previous ones. The transformer layers repeatedly refine token representations, so that the final layer can output a probability distribution over the vocabulary. By iteratively applying this process, the model generates text that is grammatically correct, semantically meaningful, and consistent with the context provided by the prompt. Transformers allow parallel processing of attention during inference, making it possible to handle long sequences and large models efficiently. Because transformers learn general language patterns, the same architecture can perform translation, summarization, reasoning, and dialogue without task-specific modifications.

3. RETRIEVAL AUGMENTED GENERATION

Retrieval-Augmented Generation is a framework created to overcome limitations by integrating retrieval-based methods with generative models. It retrieves relevant information from external sources and utilizes this data to produce contextually appropriate responses that are more accurate. The integration of generative AI models with RAG retrievers is transforming the field with remarkable efficiency and capability. A notable advantage of RAG is its adaptability, as it can be applied to various types of data, including text and images. This versatility makes RAG systems a reliable and effective tool for enhancing the capabilities of generative AI. [49]

The Retrieval-Augmented Generation method is employed to improve large language models with domain-specific documents. Initially, the documents are divided into chunks. Subsequently, the underlying meaning of each chunk is transformed into vector form using transformer-based embedding models. These indexed vectors are then stored in a vector database. To enhance the retrieval, our application utilizes hybrid search. It means that the retrieval phase uses vector search and keyword search to retrieve the data from the database. Data is then re-ranked and fed to LLM with instructions and the original query. Hybrid search is illustrated below.

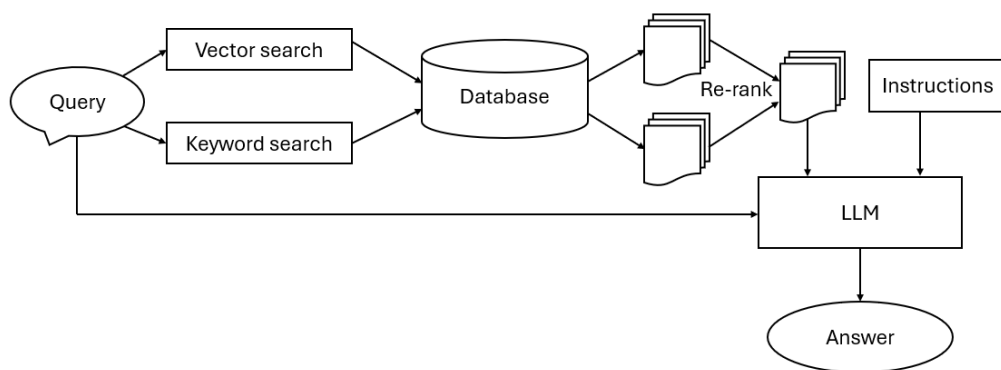


Figure 3.1. Hybrid search utilizes keyword and vector search to improve retrieval of the application.

In this chapter, we will describe the steps involved in constructing a Retrieval-Augmented Generation application. By understanding the underlying principles of RAG applications, we can systematically assess and optimize the performance of our application. In RAG

applications there are four key domains: data, storage, retrieval, and generation [49].

A Retrieval Augmented Generation application relies on the dataset from which it derives answers. Key considerations regarding the data include its storage location, its reliability, and if there are any issues with security, privacy or copyrights [49]. In this thesis, we are using public data that includes laws and public guidelines on how the specific laws are applied. It needs to be decided where the data is stored before and after processing it. In our approach, we have raw documents in a blob storage and processed data is stored as indexes in the vector database.

Retrieval is the phase wherein RAG application gets the context for the query. There are a lot of RAG frameworks that can be utilized and many parameters that can be tuned for better outcome. In the chapter 6 we will consider parameter optimization for RAG. In the generation, the AI model fitted to the application needs to be chosen and it will get the context, original query and additional information as a system prompt. In RAG application the context given to the LLM is chunk or multiple chunks of the dataset. Next we will discuss how the chunks are constructed from the data.

3.1 Chunking

In retrieval augmented generation, chunking refers to breaking down input documents and information into smaller units of text, called chunks. Determining the optimal chunk size is essential for handling external documents effectively. The initial step involves dividing them into smaller segments to facilitate the extraction and encoding of detailed features [63]. When including content to RAG model, we can expect different outcomes based on whether the content is brief (such as sentences) or lengthy (paragraphs or whole documents). This phenomenon occurs due to behavior of embedding model. Embedding model takes a whole chunk as an input and creates a vector from its semantic features. If the chunk is large, there is more context for every token in it and the final vector in the database will be different compared to a smaller chunk that won't have as much context.

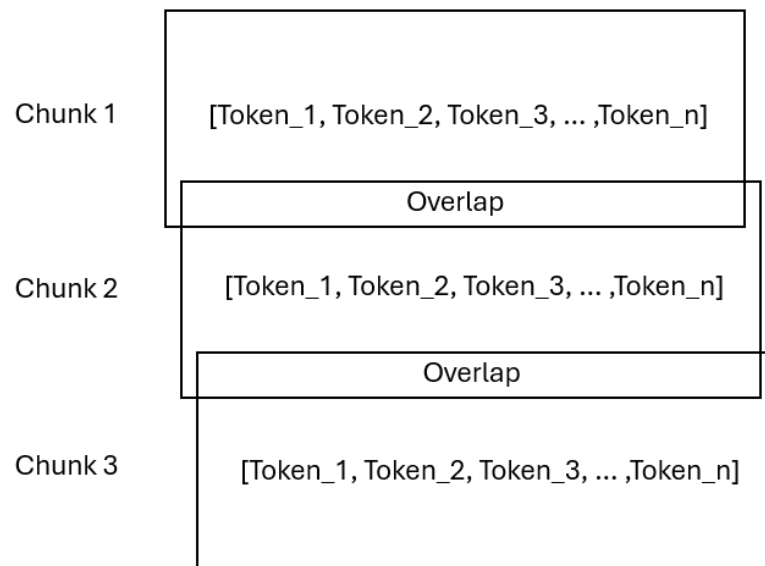


Figure 3.2. *Chunking refers to splitting the document into smaller chunks of tokens.*

When prompting the model, the query will be embedded, and the resulting vector will be compared to the ones in the vector database. The context is retrieved from the database and will be fed to LLM with the query that generates the answer. Hence, the selection of the size of the chunk must be matched to the use of the model. For question answering purposes, smaller chunk sizes return more likely more precise answers, because the vector in the database does not contain extra information that can cause hallucination. For summarizing tasks, larger chunk sizes are preferred, since the vectors will have more context. [59]

When selecting the chunking strategy, the nature of the chunked text should also be considered. There is a difference if the information stored in the database is a set of long documents, such as articles, or shorter content, such as messages. If the chunk size is large, there may be a lot of unrelated content in the same chunk. This distorts the semantic meaning of the chunk, and the answer might end up being incorrect. [55][59]

Determining optimal chunk size can be difficult, and there are many optimization techniques introduced in [63]. In this thesis we will use fixed size chunks and we use our optimization method to find the best chunk size for the application. Along the chunk size optimization, we optimize the number of chunks provided. The optimization is considered in the chapter 6. Configurable parameters are chunk size and overlap. Chunk size refers to maximum size of the chunk. It can be smaller, since the algorithm used attempts to avoid truncating a sentence meaning it terminates at sentence-ending punctuation, such as a period, exclamation point, or question mark, assuming the language has sentence-ending punctuation.

3.2 Context Retrieval

In this study, we employ a hybrid search strategy for context retrieval. This approach involves retrieving context chunks through both keyword and vector search, wherein the query is vectorized and compared with vectorized data stored in the database. To facilitate the conversion of data into vector format, embedding models are utilized. Additionally, an algorithm is required to compare the query vector with the vectors in the database. Once the appropriate document chunks are identified, a re-ranking process is necessary due to the hybrid search strategy. This section introduces the embedding models, the keyword search methodology, vector search algorithms and re-ranking.

In a RAG model, the indexed documents are stored in a vector database. The indexes are stored there as high-dimensional vectors that contain the context of the chunk.

Within the RAG pipeline, another crucial component is the search index, which stores the vectorized content obtained in the preceding step. The query vector needs to be compared with the document chunk vectors in the database so that the most similar one can be found. A common implementation is to calculate the Euclidean distance between the query vector and the vectors of all the content chunks [15]. In this thesis, we are using cosine similarity as a similarity metric.

Cosine similarity measures similarity based on the angle between the vectors rather than their length. It is calculated as the dot product of the feature values, normalized by the feature vector lengths. A key advantage of cosine similarity is its independence from vector magnitude, which makes it insensitive to document length. [15]

The cosine similarity between a query q and a document x_i is given by:

$$\text{Cos}(q, x_i) = \frac{\sum_{f \in F} q_f \cdot x_{if}}{\sqrt{\sum_{f \in F} q_f^2} \cdot \sqrt{\sum_{f \in F} x_{if}^2}}, \quad (3.1)$$

where the symbol f refers to the features in the feature space F . Here, f is the dimension of the vector. This equation represents the dot product of the two vectors, normalized by the lengths of the feature vectors. It is widely used in text analytics because it effectively captures the orientation of vectors in high-dimensional space.

We can calculate k -nearest neighbors (k -NN) by selecting the k closest document vectors to the query vector using cosine similarity [15]. In many applications, finding the exact nearest neighbors is not always essential. In RAG applications, it is often sufficient to find items that are reasonably similar rather than the most similar ones, as this can lead to unexpected and pleasant discoveries. Similarly, in k -NN classification, neighbors that are close, but not the absolute closest, are likely to belong to the correct class. The common strategies for Approximate k -NN are Locality Sensitive Hashing Random Pro-

jection Trees [15], and the Hierarchical Navigable Small World algorithm (HNSW) that is used in our architecture. The latter is a highly efficient method for the search for the approximate nearest neighbor (ANN) in high-dimensional spaces [33]. HNSW outperforms previous modern approaches and scales logarithmically even in high-dimensional data [33]. In HNSW, the efConstruction parameter controls the size of the dynamic candidate list during the index construction phase. Similarly, the efSearch parameter determines the size of the candidate list used during querying. Increasing the efSearch value improves search accuracy, but also leads to slower query times due to the larger search space [33].

Along with vector search, hybrid search retrieval also utilizes keyword search. This is done using the BM25 algorithm. It is a ranking function employed by search engines to estimate the relevance of documents to a given query. BM25 is a bag-of-words retrieval model that scores documents based on frequency of query terms, independent of their positional proximity. [48]. Given a query Q that contains keywords $q_1, \dots, q_n$, the BM25 score of a document D is [22]:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (3.2)$$

where: $f(q_i, D)$ is the frequency of keyword q_i in document D , $|D|$ is the length of document D in words, avgdl is the average document length in the text collection, k_1 and b are free parameters, typically $k_1 \in [1.2, 2.0]$ and $b = 0.75$, $\text{IDF}(q_i)$ is the inverse document frequency weight of query term q_i [22]. This component of the formula evaluates how frequently a term appears across all documents and assigns lower weight to terms that are commonly used. It is computed as:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right) \quad (3.3)$$

where N is the total number of documents in the collection and $n(q_i)$ is the number of documents containing q_i .

The IDF component is derived from the Binary Independence Model and can be interpreted from information theory. If a query term q appears in documents $n(q)$, the probability of a random choice document D containing q is $\frac{n(q)}{N}$. The information content of the message "D contains q" is [48]:

$$-\log \left(\frac{n(q)}{N} \right) = \log \left(\frac{N}{n(q)} \right) \quad (3.4)$$

For two query terms q_1 and q_2 that occur independently, the probability that both terms appear in a document D is $\frac{n(q_1)}{N} \cdot \frac{n(q_2)}{N}$, and the information content is [48]:

$$\sum_{i=1}^2 \log \left(\frac{N}{n(q_i)} \right) \quad (3.5)$$

This is reflected in the IDF component of BM25 [48]. Combining Vector search and keyword search is known to have better results than just vector search [53] [4]. Next, we will discuss how to combine these results as one set of data.

The hybrid search retrieves two individual sets of context chunks using a vector search and keywords. The results are re-ranked and listed using Reciprocal Rank Fusion (RRF). The Reciprocal Rank Fusion method is a straightforward approach to combining document rankings from various retrieval systems. RRF consistently outperforms individual systems and the traditional Condorcet Fuse method [14]. It ranks documents using a simple scoring formula. Given a set D of documents to be ranked and a set R of rankings, each being a permutation of 1 to $|D|$, the RRF score for a document $d \in D$ is calculated as [14]:

$$\text{RRFscore}(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)} \quad (3.6)$$

where $k = 60$ and $r(d)$ is the position of the document in the list.

At this point, there are results ranked from BM25 text based search and from the vector search, combined into RRF scores. To reach final list of documents we utilize Semantic search to improve relevancy of the results. The key word based results can contain irrelevant document chunks that can be filtered out at this step. Semantic search is evolving from traditional keyword-based ranking to a Transformer-based semantic ranking engine that understands the underlying meaning of text [32]. It returns a caption of the document chunk and compares it to the original query. Captions are evaluated based on their conceptual and semantic alignment with the given query [35]. Then the semantic ranker will add semantic scores to the original search scores from RRF. Added scores range from 4 to 0, with higher scores indicating greater relevance. Here are the scores and definitions by Microsoft [35]:

Table 3.1. Document Relevance Scores. Table adapted from [35].

Score	Meaning
4.0	The document is highly relevant and fully answers the question, though it may include some unrelated text.
3.0	The document is relevant but missing some details for a complete answer.
2.0	The document is relatively relevant, providing a partial answer or addressing only certain aspects of the question.
1.0	The document is related to the question and answers a small part of it.
0.0	Irrelevant document.

The semantic scores are added to the RRF scores and the highest additive scores are finally returned as the context. The semantic ranker will return the number of documents specified using Top-N parameter. It will be optimized in the later chapters.

3.3 Answer Generation

Once the context chunks that best match the prompt have been selected, they will be fed to the LLM along with the original prompt and role information. The set is called system prompt and it consists of a set of instructions, guidelines, and contextual information for the language model [42]. Hence, the system prompt is the guideline for the LLM on how to utilize retrieved text and the original prompt. Using these three components, LLM creates the final answer to the original question.

When the application is queried, the embedding model vectorizes the query in the same way as the document chunks. The query vector can then be compared with the document vectors in the vector database. The application selects the best matching document vectors to provide context for a large language model. The LLM receives the query, role information and the document chunks as context and generates an answer.

In the answer generation phase, various decoding hyperparameters significantly influence the diversity, coherence, and relevance of the outputs. Key among these are temperature, top-p (nucleus sampling), and parameters governing contextual strictness. Temperature regulates how random the model's predictions are. A low temperature, for example 0.2, creates more deterministic, conservative responses by sharply concentrating probability mass on high-likelihood tokens. In contrast, higher values, like over 0.8, increase entropy, allowing for more diverse and creative outputs. Studies have shown that this tradeoff directly affects both the creativity and coherence of generated text [38].

Top-p sampling refines this further by concentrating only on the smallest subset of words whose cumulative probability exceeds a given threshold p (commonly 0.8 to 0.95), dynam-

ically reducing the sampling space based on model confidence. This technique allows for better control over variability without introducing the instability of high-temperature sampling [64]. Contextual strictness, although less standardized, refers to how rigidly the model adheres to provided prompts or system instructions. This can be operationalized through constraints during decoding or prompt-engineering techniques, and influences whether the model extrapolates creatively or remains confined to the input. Adjustments to such parameters are crucial in sensitive domains like legal or scientific reasoning where hallucinations are less tolerated [61]. These parameters are important when balancing between creativity and factual reliability.

4. EVALUATING A RAG APPLICATION

To assess the responses retrieved by the RAG application, a set of predefined questions is required. In addition, for some metrics we need to have a right answer to the question. The right answer is called ground truth (GT) in the text so it will not be mixed with the answers given by artificial intelligence. The evaluation can be done using right metrics to achieve the evaluation criteria. Next, evaluation metrics are introduced to understand how to classify answers as right or wrong.

To gain a comprehensive understanding of the RAG application, it is essential to conduct an analysis of the various phases of its architecture. This examination will help identify the areas where the application performs well. In a RAG application, the model's ability to accurately retrieve the appropriate context for the Large Language Model to reference is critical. Numerous methodologies exist to assess the retrieval phase of this approach and they are covered in the section 4.3. Additionally, it is important to evaluate how the LLM creates the final response using both the prompt and the retrieved context. Evaluation of the generation phase is discussed in chapter 4.4.

4.1 Evaluation Framework

The need for an evaluation framework appeared from a business requirement. From the customer's perspective, it is crucial to assess whether modifications in the application setup positively impact the outcomes. To address this need, we designed an evaluation framework that is repeatable and customizable for various business requirements and can be applied across diverse AI applications.

We decided to use RAGAS metrics as the basis for our evaluation framework. The RAGAS library offers an extensive set of metrics to assess the outcomes of an AI application. The metrics used are introduced more closely in chapters 4.3 and 4.4.

In the evaluation framework, we have a set of questions and ground truth for every question. In our approach, we have two phases, collecting the data and evaluating the responses. First we collect the data by prompting the AI application using all the questions and get AI formed answers that will be saved for every question. We also save the context retrieved for each question so that we can use it in the evaluation phase. Then in

the evaluation phase, the responses given by the AI application is assessed with metrics provided by the RAGAS library. The results can be visualized with Python and saved for further analysis.

In the upcoming sections, we introduce our evaluation metrics. First, we will focus on statistical analysis.

4.2 Statistical Analysis

Let us consider a classification problem where we have only two classes and every instance I is mapped to one element of the set $\{p, n\}$, where p is positive and n is negative [24]. In classification problems, we evaluate the performance of a model using two conditions: predicted and actual outcomes. If the model predicts a positive outcome and the actual outcome is also positive, it is counted as a true positive. If the model predicts a negative outcome, but the actual outcome is positive, it is counted as a false negative. If the model predicts a positive outcome, but the actual outcome is negative, it is false positive. Lastly, if the model predicts a negative outcome and the actual outcome is also negative, it is true negative. These categories can be visualized using a confusion matrix.[24]

Confusion matrix is defined in [51] and is following.

Definition 4.1. Confusion Matrix

A confusion matrix C is a table where $C_{i,j}$ represents the number of observations that actually belong to group i but are predicted to be in group j . In a 2×2 confusion matrix form, confusion matrix is defined as:

$$C = \begin{bmatrix} \text{True Positives (TP)} & \text{False Negatives (FN)} \\ \text{False Positives (FP)} & \text{True Negatives (TN)} \end{bmatrix}$$

Using confusion matrix, several evaluation metrics can be defined. Precision and recall are defined as follows.

Definition 4.2. Precision and recall

Recall R measures the percentage of actual positive cases that are accurately identified as positive. In contrast, precision P represents the percentage of predicted positive cases that are truly positive. Using confusion matrix, precision and recall can be formatted as

$$R = \frac{TP}{TP + FN} \tag{4.1}$$

$$P = \frac{TP}{TP + FP}, \tag{4.2}$$

where TP is true positive, FP is false positive, TN is true negative, and FN is false negative[39].

The F-measure evaluates the performance of a classification model by combining precision and recall. F-measure is defined as:

Definition 4.3. F-measure

The F-measure is computed as the harmonic mean of precision and recall.

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R} \quad (0 \leq \beta \leq +\infty), \quad (4.3)$$

where β is relative importance factor given to precision and recall [13]. If we set β to one, we get F_1 -measure that is commonly called F-measure and denoted as:

$$F = \frac{2PR}{P + R}, \quad (4.4)$$

Where P is precision and R is recall [52].

The F measure is used widely in our evaluation metrics that are presented in the following chapters. First, we outline the methods for evaluating the retrieval phase of the RAG application. Next, we discuss the evaluation of the generation phase. Finally, we explain how these evaluation metrics are used to assess the overall effectiveness of the RAG application.

4.3 Retrieval Evaluation

Evaluating the retrieval phase of the Retrieval-Augmented Generation model is critical for ensuring the model's effectiveness. This phase involves assessing the model's ability to accurately retrieve relevant context, which is essential for the LLM to generate contextually specific and accurate responses. Various methodologies can be used to measure the performance of the retrieval phase, providing information on the overall efficiency and reliability of the RAG model.

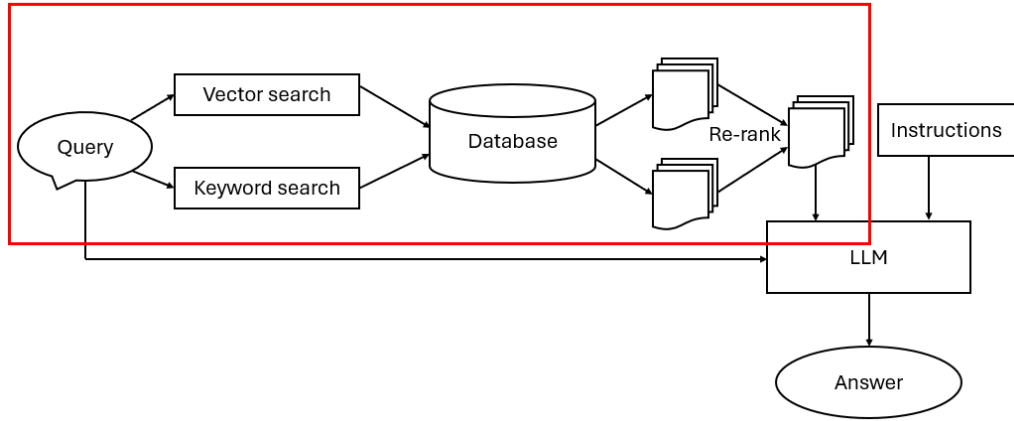


Figure 4.1. Figure of the retrieval phase of the RAG model. It is pointed out that there are many steps in the phase that can affect to the final contexts provided to large language model.

Many of the Ragas metrics use LLM as a judge. This means that the metrics are using LLM capabilities in classification tasks so that answers can be efficiently evaluated. Next, metrics used to evaluate the retrieval phase of the AI application are introduced. Various metrics are available to assess different aspects of retrieval-augmented generation.

Definition 4.4. Context Precision

Context precision measures ratio of how many of the retrieved chunks are relevant. Let the total number of chunks in the context be denoted as K , and each chunk as $k \in K$. Context precision is calculated as the average precision at chunk k for each chunk. Precision at k represents the proportion of relevant chunks at rank k to the total chunks at rank k . Context precision can be calculated as follows [19]:

$$\text{Context Precision} = \frac{\sum_{k=1}^K (P_k \times v_k)}{K \times v_k}, \quad (4.5)$$

where P is precision defined in the equation (4.2) and $v_k \in \{0, 1\}$ is a relevancy indicator of whether a chunk is relevant or irrelevant. RAGAS metric creates a prompt that asks whether the context was useful in arriving at the given answer. The prompt includes the question, context, and answer. Then, the large language model evaluates the prompt and

generates a response. The response includes a reason for the verification and a binary verdict $v_k \in \{0, 1\}$ [29].

Context recall is calculated using user input, ground truth, and retrieved contexts. Its values range from 0 to 1. The metric uses user input to retrieve the context and then the ground truth as a proxy for reference contexts, simplifying the process since annotating reference contexts is time-consuming. To estimate context recall, the ground truth is divided into claims, each analyzed to see if it can be attributed to the retrieved context[20]. It can be defined as:

Definition 4.5. Context Recall

Let $\mathcal{G}$ denote the set of ground truth claims $c \in \mathcal{G}$, and let $\mathcal{C}$ denote the retrieved context. Context Recall is defined as the proportion of ground truth claims that can be attributed to the retrieved context:

$$\text{Context Recall} = \frac{|\{c \in \mathcal{G} \mid c \text{ is attributable to } \mathcal{C}\}|}{|\mathcal{G}|}. \quad (4.6)$$

Here, $|\{c \in \mathcal{G} \mid c \text{ is attributable to } \mathcal{C}\}|$ represents the number of ground truth claims supported by the retrieved context, and $|\mathcal{G}|$ denotes the total number of claims in the ground truth. Ideally, all claims in the ground truth are attributable to the retrieved context, in which case the context recall equals 1. The employed large language model determines whether a claim can be attributed to the retrieved context.

Using these metrics we can evaluate the retrieval phase of the application. This ensures that the claims in the answers given are in fact found from the context we are using.

4.4 Generation Evaluation

Evaluating the generation phase of the RAG model is essential for understanding its overall performance. This phase focuses on assessing how effectively the Large Language Model utilizes the retrieved context and the given prompt to generate accurate and coherent responses. By employing various evaluation methodologies, we can gain insights into the quality and reliability of the generated outputs, thereby ensuring the robustness of the RAG model.

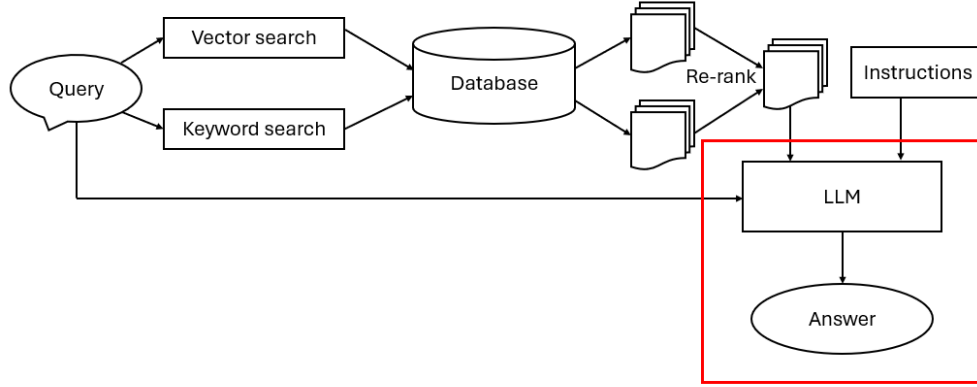


Figure 4.2. Figure of the generation phase of the RAG model.

Answer semantic similarity is a metric that captures the semantic meaning of the AI given answer and the ground truth and compares the semantic meaning. To simplify, there is a high dimensional space of understanding behind embedding models. Every sentence and word can be represented as vectors. The mechanism is explained previously in the chapter 2. If the semantic meanings of the sentences or paragraphs are similar, the vectors compared have a small angle between them. This can be used as a method to evaluate whether the answer given by RAG model and the ground truth are understood as similar by the embedding model.

Definition 4.6. Answer Semantic Similarity

Answer semantic similarity first vectorizes the ground truth using the specified embedding model and the generated answer with the same model. Then it computes the cosine similarity between the two vectors introduced in section 3.2. Applying the ground truth and answer into the cosine similarity equation it is noted as follows:

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^n G_i A_i}{\sqrt{\sum_{i=1}^n G_i^2} \cdot \sqrt{\sum_{i=1}^n A_i^2}}, \quad (4.7)$$

where G_i is i th component of the Ground truth vector $\mathbf{G}$ and A_i is the i th component of the answer vector $\mathbf{A}$.

Answer Correctness is another metric that is used to evaluate the results. It is introduced in Ragas' documentation [18] and is defined as follows.

Definition 4.7. Answer Correctness

Answer correctness evaluates how accurately a generated response matches the expected or ground truth answer. It is calculated using F-measure introduced in equation (4.4) and semantic similarity. The proportions of these two metrics can vary depending on an application. Semantic similarity is calculated by vectorizing ground truth answer and the generated answer using the same embedding model and computing the cosine similarity between these two vectors. F-measure is calculated as follows:

$$F = \frac{2PR}{P + R}, \quad (4.8)$$

where P is precision and R is recall. That can be expressed as

$$F = \frac{TP}{TP + 0.5(FP + FN)}, \quad (4.9)$$

where TP is true positive, FP is false positive, FN is false negative, and the labels are calculated using LLM. TP equals claims that are present in ground truth and in answer. FP means claims that can be found from the generated answer but not from the ground truth. FN is the label for claims that are in the ground truth but not in the answer.

Using the metrics introduced in this chapter, we can assess different stages and aspect of the rag model and define whether the answer is correct or not. Many of the metrics use LLM to define the accuracy of the answer. In some cases the LLM can be wrong, but by combining many different metrics to evaluate an answer we have a high probability that evaluation is correct. All of the four metrics introduced in this chapter are normalized. The total sum of the metrics is used to guide Bayesian optimization.

5. BAYESIAN OPTIMIZATION

In this study, we employ Bayesian optimization to tune the chunk size and the number of chunks provided as input to the large language model. Bayesian optimization is a category of optimization techniques that leverage machine learning to address the problem of maximizing $f(x)$ over the domain $x \in \mathcal{X}$ [25]. It is a nonlinear optimization technique where the algorithm selects the next point to explore based on a distribution over functions $P(f)$, such as a Gaussian process or other surrogate models. The selection is guided by an acquisition function α . This method benefits from retaining all observations in the posterior distribution $P(f \mid \mathcal{D})$, which is sequentially updated using a nonparametric model. Each observation enhances the understanding of the function across the input space due to the model’s spatial correlation induced by the kernel, reducing the number of iterations needed compared to other nonlinear optimization algorithms. However, updating the posterior and maximizing the acquisition function increases the cost per sample. Therefore, Bayesian optimization is typically used for optimizing expensive target functions, which may be multimodal or lack a closed-form expression. The quality of the prior and posterior information about the surrogate model is crucial, as it can significantly reduce the number of evaluations required to achieve optimal performance. First, we need to define the objective function $f : \mathcal{X} \rightarrow \mathbb{R}$ that will be optimized. In this study, the objective function f is defined as a sum of multiple evaluation metrics reflecting retrieval quality and answer performance. Specifically,

$$\begin{aligned} f(x) = & w_r \cdot \text{ContextRecall}(x) + w_p \cdot \text{ContextPrecision}(x) \\ & + w_c \cdot \text{AnswerCorrectness}(x) \\ & + w_s \cdot \text{AnswerSimilarity}(x), \end{aligned} \tag{5.1}$$

where x denotes the RAG parameters being optimized, and w_r , w_p , w_c , and w_s are weighting coefficients controlling the relative importance of each metric. In this study weighting coefficients $w_i = 1$. The objective function produces a single scalar value, which enables the use of standard Gaussian Process.

The objective function is often expensive to evaluate, meaning each evaluation might take a significant amount of time or resources. Bayesian optimization uses a surrogate model to approximate the objective function.

Let us begin by introducing Bayes' theorem, a fundamental principle that provides a mathematical rule for reversing conditional probabilities, enabling us to infer the likelihood of a cause given an observed effect.

Theorem 5.1 (Bayesian Theorem). $P(A | B)$ is the posterior probability of A given B and it is defined as:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)} \quad (5.2)$$

Where:

$P(A | B)$ is the probability of event A occurring given that B is true.

$P(B | A)$ is the probability of event B occurring given that A is true.

$P(A)$ and $P(B)$ are the probabilities of events A and B occurring independently.

Proof. By definition, the conditional probability of A given B is:

$$P(A | B) = \frac{P(A \wedge B)}{P(B)} \quad (5.3)$$

Similarly, the conditional probability of B given A is:

$$P(B | A) = \frac{P(A \wedge B)}{P(A)} \quad (5.4)$$

From the definition of conditional probability, we can express the joint probability $P(A \wedge B)$ as:

$$P(A \wedge B) = P(A | B) \cdot P(B) \quad (5.5)$$

and

$$P(A \wedge B) = P(B | A) \cdot P(A) \quad (5.6)$$

Since both expressions represent the same joint probability $P(A \wedge B)$, we can set them equal to each other:

$$P(A \wedge B) \cdot P(B) = P(B | A) \cdot P(A) \quad (5.7)$$

To isolate $P(A | B)$, divide both sides by $P(B)$:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)} \quad (5.8)$$

□

Bayesian optimization is not a single algorithm, but rather a conceptual framework for

optimization based on Bayesian inference, from which a wide range of algorithms have been developed [27]. Optimization involves a series of decisions: at each step, we must determine where to observe next and decide whether to stop based on the result. Since these outcomes are dictated by the system being studied and lie beyond our control, the effectiveness of the optimization process depends entirely on making good decisions [27]. The Bayesian approach uses probability and inference to manage uncertainty in optimization, treating the objective function as a random function shaped by prior knowledge and observed data. This evolving belief actively guides decisions, helping to evaluate where to sample next based on expected outcomes.

We consider a real-valued objective function defined over a domain $\mathcal{X}$, denoted as

$$f : \mathcal{X} \rightarrow \mathbb{R}.$$

No specific assumptions are made about the structure of $\mathcal{X}$; it may be a Euclidean space or a more complex set, such as one composed of structured objects. The goal of optimization is to identify a point $x^* \in \mathcal{X}$ that achieves the global maximum of the function [56, 9]:

$$x^* \in \arg \max_{x \in \mathcal{X}} f(x), \quad f^* = \max_{x \in \mathcal{X}} f(x) = f(x^*). \quad (5.9)$$

We start with an initial dataset

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n, \quad (5.10)$$

which may be empty, and expand it incrementally through a series of observations [9]. Here, each observation y_i corresponds to the value of the aggregated objective function,

$$y_i = f(x_i), \quad (5.11)$$

that is computed using the weighted combination of recall, precision, answer correctness, and answer similarity. At each step, an optimization policy reviews the current data and selects a point $x \in \mathcal{X}$ for the next evaluation. This produces a new observation

$$y = f(x) + \varepsilon, \quad (5.12)$$

where

$$\varepsilon \sim \mathcal{N}(0, \sigma_n^2), \quad (5.13)$$

which we add to the dataset.

We assume that observations are generated by a stochastic process influenced by the objective function. Specifically, the objective function f is modeled as a random function,

and for a fixed input x , the function value $f(x)$ is a random variable [47]. The observed value y at point x is modeled as a noisy realization of this latent function value.

In our optimization model, the distribution of each observation is governed by a specified observation model $p(y \mid x, f(x))$. A prior distribution over functions $p(f)$ represents our initial beliefs before any data is observed. Once an observation y is obtained, we update our belief about the objective function using Bayes' theorem, resulting in the posterior distribution

$$p(f \mid \mathcal{D}_n). \quad (5.14)$$

A Gaussian Process (GP) generalizes the finite-dimensional multivariate Gaussian distribution by replacing the mean vector and covariance matrix with corresponding functions defined over a continuous domain [47].

Formally, a Gaussian Process prior over the objective function is defined as

$$f \sim \mathcal{GP}(m(x), k(x, x')), \quad (5.15)$$

where $m(x) = \mathbb{E}[f(x)]$ is the mean function and $k(x, x')$ is a positive semidefinite covariance function [47].

The Gaussian Process gives a posterior distribution over functions after observing the data $\mathcal{D}_n$. At a new point x , the predictive distribution of the latent function value is

$$f(x) \mid \mathcal{D}_n \sim \mathcal{N}(\mu(x), \sigma^2(x)). \quad (5.16)$$

The Expected Improvement (EI) is an acquisition function that guides where to evaluate the function next [30].

Definition 5.2. Expected Improvement Let

$$y^+ = \max_{i=1, \dots, n} y_i$$

be the best observed function value so far. The improvement at a candidate point x is defined as

$$I(x) = \max(f(x) - y^+, 0).$$

The Expected Improvement is the expectation of this improvement under the posterior distribution of the GP model [30]:

$$\text{EI}(x) = \mathbb{E}[\max(f(x) - y^+, 0)]. \quad (5.17)$$

If

$$f(x) \sim \mathcal{N}(\mu(x), \sigma^2(x)), \quad (5.18)$$

the EI admits the closed-form expression [30]:

$$\text{EI}(x) = (\mu(x) - y^+) \Phi\left(\frac{\mu(x) - y^+}{\sigma(x)}\right) + \sigma(x) \phi\left(\frac{\mu(x) - y^+}{\sigma(x)}\right). \quad (5.19)$$

Here, ϕ is the probability density function of the standard normal distribution and Φ is its cumulative distribution function. EI provides a principled way to balance between exploitation and exploration [30, 9]. It prefers points with high predicted mean $\mu(x)$ and points with high uncertainty $\sigma(x)$.

In this thesis, we model the unknown objective function $f : \mathcal{X} \rightarrow \mathbb{R}$ using a Gaussian Process:

$$f(x) \sim \mathcal{GP}(0, k(x, x')). \quad (5.20)$$

We use the squared exponential (RBF) kernel [47]:

$$k_{\text{RBF}}(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2l^2} \|x - x'\|^2\right), \quad (5.21)$$

where σ_f^2 is the signal variance and l is the length-scale parameter [47].

Given observations

$$\mathcal{D}_n = \{X, \mathbf{y}\} = \{x_1, \dots, x_n, y_1, \dots, y_n\}, \quad (5.22)$$

and assuming Gaussian observation noise

$$\varepsilon \sim \mathcal{N}(0, \sigma_n^2), \quad (5.23)$$

the GP predictive distribution at a new point x_* is

$$f(x_*) \mid \mathcal{D}_n \sim \mathcal{N}(\mu(x_*), \sigma^2(x_*)). \quad (5.24)$$

The predictive mean is

$$\mu(x_*) = k(x_*, X) [K + \sigma_n^2 I]^{-1} \mathbf{y}, \quad (5.25)$$

and the predictive variance is

$$\sigma^2(x_*) = k(x_*, x_*) - k(x_*, X) [K + \sigma_n^2 I]^{-1} k(X, x_*), \quad (5.26)$$

where $K \in \mathbb{R}^{n \times n}$ is the Gram matrix with entries $K_{ij} = k(x_i, x_j)$, and $k(x_*, X)$ is the vector of covariances between x_* and each training point. In practice, a small additional jitter term may be added to the diagonal of K for numerical stability [47].

To select hyperparameters

$$\theta = \{\sigma_f^2, l, \sigma_n^2\}, \quad (5.27)$$

we maximize the log marginal likelihood [47]:

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2} \mathbf{y}^\top [K + \sigma_n^2 I]^{-1} \mathbf{y} - \frac{1}{2} \log |K + \sigma_n^2 I| - \frac{n}{2} \log 2\pi. \quad (5.28)$$

In practice, this optimization is performed using gradient-based optimizers with multiple restarts. The surrogate model is a Gaussian Process that provides both predictive mean and uncertainty, and whose behavior is fully determined by the kernel function and its hyperparameters.

6. OPTIMIZING RAG PARAMETERS

In this chapter, the technical architecture and parameters are introduced. First we will start the chapter by reviewing related work.

6.1 Related Work

Artificial intelligence is currently a fascinating and rapidly advancing field. In recent years, there has been a notable increase in research focused on evaluating and optimizing AI applications. Many of these studies concentrate on improving the retrieval phase, which is important for improving the performance of AI systems [53], [62], [1].

Additionally, there has been considerable work on refining querying methods to make AI systems more efficient and accurate. Chi-Min Chan et al.[12] have explored various strategies for learning to refine queries, while Wang et al. [60] have investigated the role of filtering in context retrieval-augmented systems.

Optimizing knowledge bases is another area that has garnered attention. Packowski et al. (2024) have studied methods to enhance the organization and accessibility of knowledge within AI systems [41]. Furthermore, researchers like Sakar and Emecki (2024) have focused on improving the run times and efficiency of AI models, ensuring they operate effectively [50].

Embedding models, which are essential for understanding and processing data, have also been a subject of comparison and evaluation. Caspari et al. (2024) have provided benchmarks for assessing the performance of various embedding models, contributing to the overall advancement of AI technology [11].

In this work, we utilize a semantic hybrid search architecture optimized by Sawarkar et al. (2024) [53]. The optimal chunk sizes have been studied by Bhat et al. (2025) [5]. They propose small chunk sizes (64-128 tokens) for fact based questions and larger chunks (512-1024 tokens) for descriptive responses. Microsoft suggests chunk size of 512 tokens in the official document [34]. Many of the features employed in this study have been researched and validated as best practices. Nowadays, Retrieval-Augmented Generation applications are relatively easy to build. The purpose of this work is to demonstrate how to evaluate the model and how the results can be used to optimize an application.

During the optimization phase, we will focus on adjusting easily modifiable parameters and highlighting their significance. The final functionality of a RAG application depends on the documents it is based on. Therefore, while the results may not be universally applicable to all applications, the optimization process is repeatable for any RAG application, and the parameters can be tailored to specific needs. The purpose of the study is to show simple adjustments that can be deployed for a RAG application for better results. Next, the setup of the application is introduced.

6.2 Test Setup

The architecture for the solution integrates Azure-based services to enable user interaction, information retrieval, and the generation of natural language responses. This architecture is designed to implement a Retrieval-Augmented Generation framework, which can facilitate the web-hosted chatbot to provide accurate and document-based answers for users.

RAG application is built in Microsoft Azure environment. The documents are stored in a blob storage. All documents relate to work accident insurance and its conditions. The documents include both legal texts and guidelines for applying the law, as well as examples of compensated work accidents. The documents are in Finnish. The documents are listed in the appendix A.

The architecture uses hybrid search presented in the chapter 3. The hybrid search enables also semantic ranker that enhances the relevancy of the context. In the retrieval phase, the number of best matching chunks (Top-N) are retrieved and used as a context to the query. Query, context and role information are fed to GPT-4o LLM model that will produce the final answer. The context is retrieved using hnsw algorithm.

The question that we are using to evaluate AI given answers are simulating real life questions related to the documents. The ground truth answers are composed with a specialist and they are formulated accurately. Both the questions and the ground truth answers, as well as the documents, are written in Finnish. In the test setup, there are 79 questions used as queries that are evaluated. In this study, ground truth answers are short, each containing a limited number of claims, generally fewer than ten.

The context documents that are stored in the blob storage contain legal text and guidelines for insurance companies on how to resolve work related accident cases. The context material contains 1168413 tokens in total. The size of the documents range from 4519 to 425059 tokens. The document sizes by token count are listed in the appendix A. The detailed parameters of the application are listed in the appendix B. Although the full context material contains over one million tokens, the evaluation does not compare ground truth claims against all claims in the context material. Instead, claim-level evaluation is

performed only on the retrieved context, whose size and number of claims are bounded but depend on the chunk size and the Top-N parameter.

The optimization program is designed to optimize parameters that influence the quality of answers generated by a language model. It works by repeatedly running a cycle where it first updates a configuration with candidate parameter values, then collects answers from the model using a fixed set of questions, and evaluates those answers to produce a performance score. This score is used as feedback for a Bayesian optimization algorithm, which selects new parameter values based on previous results to efficiently search for the best combination. The optimizer is guided by the total score which is a sum of selected evaluation metric scores. The process continues for ten iterations, and then until it hits early stopping ($n = 3$). The early stopping parameter has been set so that if the total score does not improve in last 3 rounds, the optimizer stops. After optimization, the program analyzes the history of parameter choices and scores, visualizes trends and relationships, and finally reports the best-performing parameter set along with its score. The Optimization workflow is illustrated below.

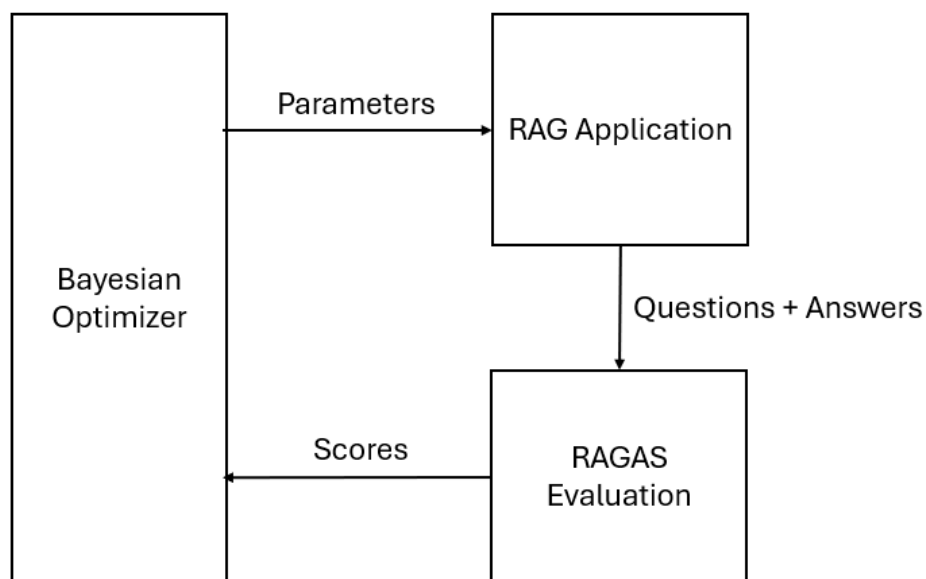


Figure 6.1. Optimization flow overview.

The optimized parameters are Top-N and chunk_size. The first parameter represents the number of document chunks retrieved and used by the RAG model to generate an answer, while the second parameter defines the size of each chunk, expressed in tokens, into which the context is divided.

6.3 Optimization

Optimizing the retrieval is widely discussed in recent literature [42][26]. In this thesis, we utilize hybrid search method and focus on optimizing the retrieval of the model. The interaction between retrieval methods and prompt design forms a challenging optimization landscape that is still mostly uncharted [42]. The optimization method introduced here can be addressed to optimize prompt design. This can be studied in the future. In our approach we are concentrating in studying the affect of the context in RAG method. The optimized parameters are shown in the table below.

Table 6.1. Test values for optimization.

Parameter	Test Values
Chunk Size	500, 2000, 5000, 10000, 15000
Top-N	1, 2, ..., 10

Chunk size optimization is limited to chunk sizes between 500 and 15000 tokens and the sizes are limited into fixed values. The overlap for each chunk size is 25 % as suggested by Microsoft [34]. Top-N parameter is also limited between one and ten. Gathering the answers and evaluating them using a large set of questions is expensive and time consuming. For this reason, the parameters have a strict bounds. Also, if the chunk size and Top-N parameters are on the top of the bound, the number of tokens in the context will have 150000 tokens. As it been shown during the simulations, a large context makes the answer processing slow and unreliable.

The optimizer was run initial 10 rounds and then using an early stopping parameter of 3. This means that if the optimizer does not find better results in 3 rounds, it stops at the best iteration. This is used because both collecting data and evaluating them are very token consuming and to manage the cost, early stopping parameter was essential.

7. RESULTS

This chapter presents a comprehensive analysis of the optimization process for the RAG models, examining the behavior of the evaluation metrics across varying parameter configurations. The interactions and trends observed in these metrics are explored, leading to the identification of the parameter settings that produce the highest overall performance. Once the results have been introduced, a discussion of their implications follows.

The optimization experiments evaluated how different retrieval configurations influence the performance of the RAG system. By varying the number of retrieved documents (Top-N) and the chunk size, the study measured their effects on four evaluation metrics: answer similarity, context recall, context precision, and answer correctness. The combined Total score was the primary indicator of the overall effectiveness of the system. The token count of the context varies from 0.086 % to 7.7 % of the total tokens available in the context. The results demonstrate that moderate chunk sizes and mid-range Top-N values generally yield the highest performance, with the best configurations achieving a balanced improvement across both recall- and precision-oriented metrics. Table 7.1 summarizes all experimental rounds and their corresponding metric values.

Table 7.1. Evaluation metrics for RAG optimization experiments.

Round	Top-N	Chunk Size	Token Count	Answer Similarity	Answer Correctness	Context Precision	Context Recall	Total
1	2	500	1000	0.8909	0.2412	0.7342	0.4615	2.3278
2	8	10000	80000	0.9049	0.2707	0.7067	0.8228	2.7051
3	7	5000	35000	0.9031	0.2759	0.7350	0.7911	2.7051
4	9	500	4500	0.8993	0.2523	0.6631	0.7115	2.5262
5	9	10000	90000	0.9038	0.2661	0.6782	0.7806	2.6287
6	4	10000	40000	0.9000	0.2800	0.7200	0.7400	2.6400
7	1	10000	10000	0.8929	0.2064	0.6709	0.4409	2.2111
8	6	500	3000	0.8977	0.2657	0.7114	0.6224	2.4972
9	6	10000	60000	0.9001	0.2665	0.6935	0.7722	2.6323
10	9	5000	45000	0.9027	0.2657	0.6991	0.8418	2.7092
11	8	2000	16000	0.9010	0.2710	0.8125	0.7679	2.7525
12	5	5000	25000	0.9026	0.2653	0.7703	0.7869	2.7251
13	6	2000	12000	0.9016	0.2510	0.8257	0.7806	2.7589
14	7	2000	14000	0.9006	0.2550	0.8093	0.7932	2.7582
15	5	5000	25000	0.9022	0.2552	0.7788	0.7679	2.7041
16	5	2000	10000	0.9020	0.2541	0.8320	0.7595	2.7476

At first the optimizer was trying to find the best scores, and after seven rounds it started to converge. The optimizer was set to stop after ten rounds if there is no improvement in last three rounds. The optimizer run sixteen rounds and the total evaluation score is presented below.

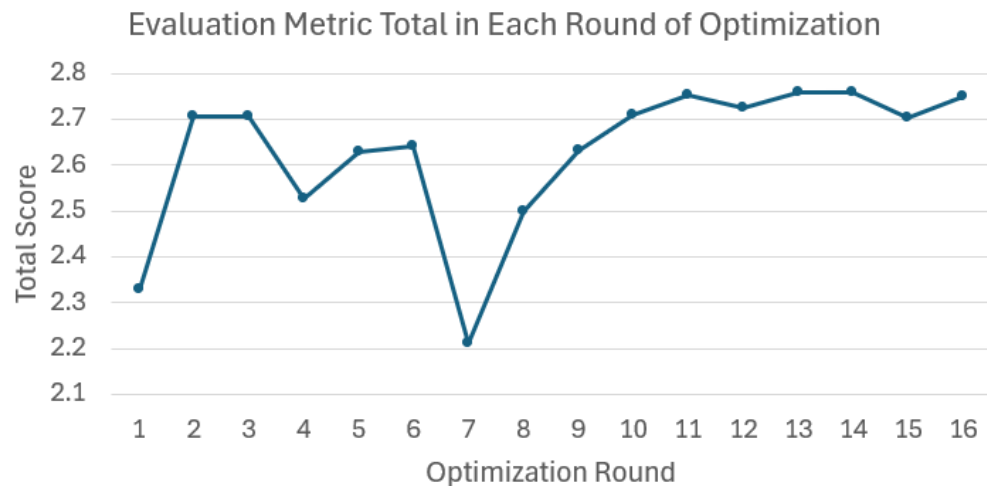


Figure 7.1. The figure shows the total scores each round of the optimization. Graph visualizes how the optimization converges between 2.7 and 2.8.

The total score used in the optimization is the sum of the separate evaluation metrics. To analyze which metric has the greatest impact on the evaluation scores, the average and standard deviation are calculated and presented in the table 7.2.

Table 7.2. Summary statistics for evaluation metrics.

	Answer Similarity	Context Recall	Context Precision	Answer Correctness
Average	0.9004	0.7276	0.7398	0.2591
STD	0.0038	0.1184	0.0573	0.0176

The standard deviation clearly indicates that Context Recall has the greatest impact on total scores. This affects the results, and the weights of the scores could be adjusted if we would want to optimize application to have specific features. For example, if the goal for the RAG model is to maximize context precision rather than recall.

The results of the parameter search reveal several meaningful relationships between the configuration of the retrieval system and the evaluation metrics. Across the sixteen rounds, Top-N documents, chunk size, and total token count show consistent patterns in how they influence answer similarity, context recall, context precision, and answer correctness, which together define the combined Total score.

An examination of the relationship between token count and the Total score shows that the connection is not linear but follows a pattern of diminishing returns. At very low to-

ken counts, such as 1,000 to 3,000 tokens in rounds 1 and 8, the Total score tends to be noticeably lower. This indicates that insufficient contextual coverage restricts recall and overall effectiveness. As token count increases into the mid-range, roughly between 10,000 and 25,000 tokens, performance consistently improves. Many of the highest-scoring configurations fall within this interval, including rounds 13, 14, 11, 12, and 16, which all achieve Total scores above 2.72.

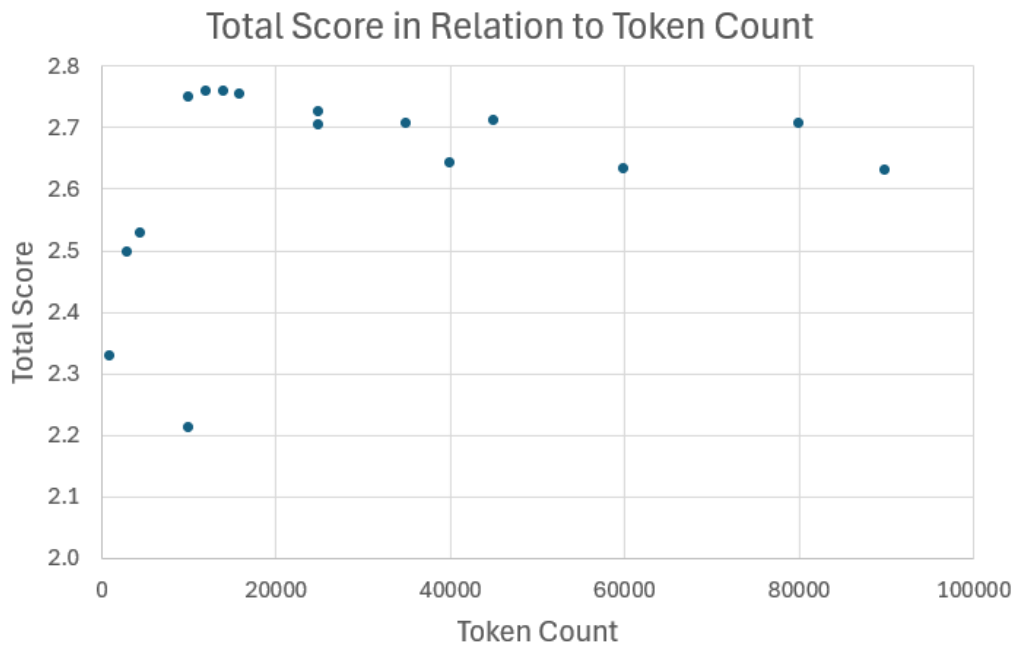


Figure 7.2. The figure visualizes the behavior of total score in relation to the token count which is the Top-N parameter multiplied by the chunk size. The best results are achieved between 10,000 and 25,000 tokens.

Interestingly, very high token counts, particularly those above 40,000 to 90,000 tokens, do not produce further improvements. This suggests that beyond a certain threshold, providing substantially more text introduces noise that weakens precision without producing a matching gain in recall. As a result, the Total score plateaus or even declines slightly. The data indicate that optimal performance occurs with moderate token counts, where there is enough information to support strong recall but not so much that irrelevant material weaken precision. The best-performing setups, with Total scores around 2.75–2.76, consistently use token counts between 10,000 and 16,000 tokens. This reinforces the idea that balanced, not maximal, context volume produces the most effective RAG behavior.

A clear trend is that extremely small Top-N values or very small chunk sizes tend to degrade recall-oriented metrics. For example, Round 1 (Top-N = 2, chunk size = 500) yields one of the lowest recall scores at 0.4615, which directly suppresses the Total score to 2.3278. Similarly, Round 7, with Top-N = 1, suffers the worst recall (0.4409) and correspondingly one of the lowest overall totals (2.2111). This indicates that providing too few documents restricts the model's access to relevant information, harming holistic perfor-

mance. This also explains the outlier in the figure 7.2 for token count 10000. In contrast, higher Top-N values, particularly between 6 and 9, consistently elevate recall into the range of 0.75–0.84. Rounds 10, 11, 13, and 14 illustrate this behavior. Context recall increases steadily in relation to Top-N documents as seen in the figure 7.3.

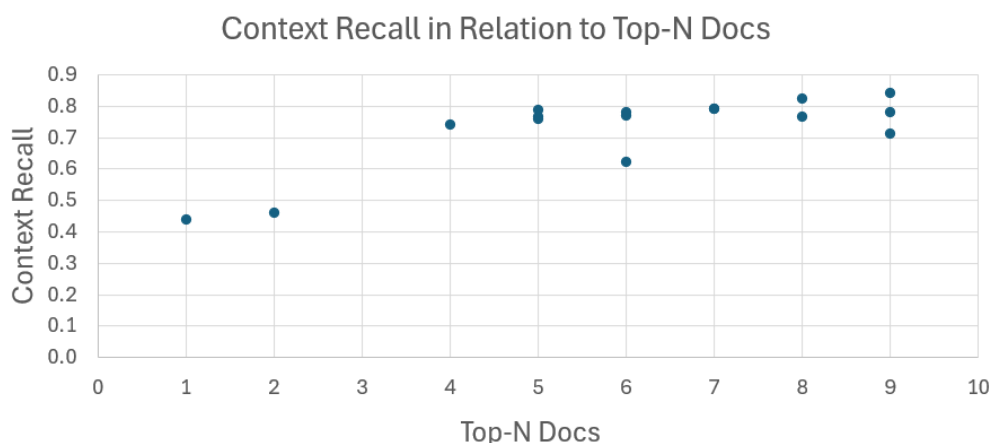


Figure 7.3. *Context Recall in Relation to Top-N documents. Context recall increases proportionally with the number of retrieved documents (Top-N). Higher document counts result in improved recall performance.*

Chunk size also contributes noticeably to retrieval performance. Extremely large chunk sizes, such as 10,000 tokens, do not necessarily outperform moderate chunk sizes. While they sometimes boost recall by exposing the model to more contiguous context, they also risk diluting precision by including unnecessary or noisy information. Rounds 2, 5, 6, and 9, all with 10,000-token chunks, produce strong but not exceptional total scores, typically between 2.62 and 2.70. The best-performing configurations tend to use chunk sizes in the range of 2,000 to 5,000 tokens. These provide a balance between sufficiently rich context and manageable granularity, which appears to preserve precision while maintaining high recall.

When the Top-N value is held constant and only the chunk size is varied, a clear trade-off emerges between context recall and context precision. Increasing the chunk size expands the amount of text contained within each retrieved segment, which in turn raises the likelihood that relevant information appears somewhere within the retrieved context. This naturally boosts recall, since the system becomes more capable of capturing the necessary evidence to answer a query. However, the larger chunks also introduce proportionally more irrelevant or loosely related material, making the retrieved context less focused. As a result, although the system retrieves more total relevant content, a smaller fraction of the retrieved text is truly pertinent to the query, leading to a decline in precision. This inverse relationship demonstrates that larger chunk sizes favor broader coverage at the expense of contextual accuracy, whereas smaller chunk sizes promote higher precision but risk missing important information and reducing recall.

Similarly, when the chunk size is fixed and Top-N is increased, the system shows the same recall–precision trade-off. Retrieving more documents improves recall by expanding the amount of potentially relevant information, but it also introduces more irrelevant content, which lowers precision. As with larger chunk sizes, increasing Top-N enhances coverage at the cost of contextual focus.

A particularly notable finding is the behavior of context precision and its relationship with chunk size. The behavior is shown in the figure 7.4. Moderate chunk sizes (2,000–5,000) tend to produce the highest precision scores, with several values exceeding 0.80. Rounds 11, 13, 14, and 16 demonstrate this trend, where precision rises to between 0.81 and 0.83. This suggests that chunk sizes in this range offer the model a more focused and less noisy retrieval space, improving the alignment of retrieved content with query relevance.

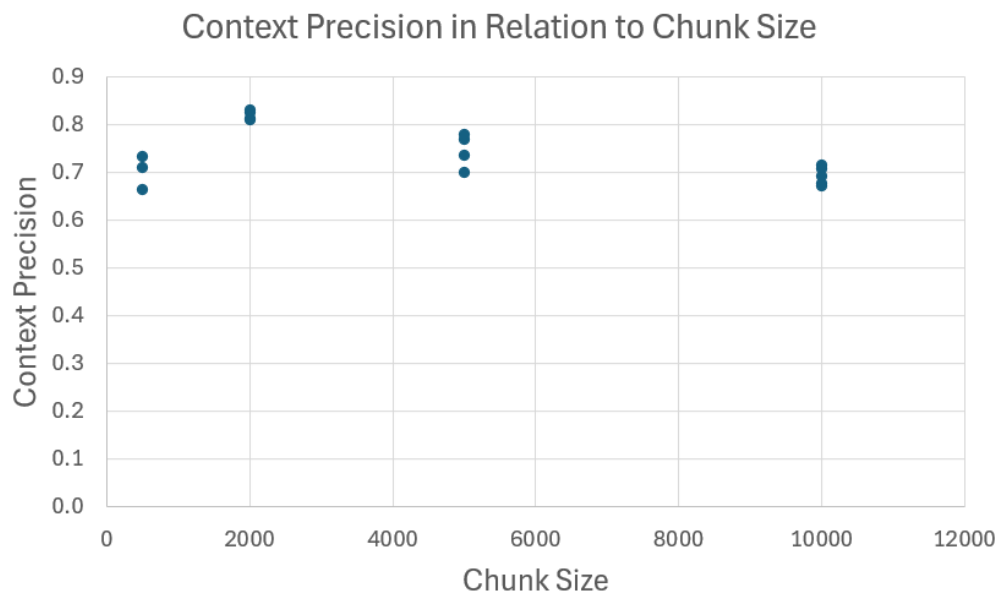


Figure 7.4. Context Precision in relation to chunk size. The most precise results are achieved at chunk size = 2000.

The total results across all rounds clearly highlight the best-performing configurations. Round 13 achieves the highest Total score at 2.7589, closely followed by Round 14 at 2.7582 and Round 11 at 2.7525. These rounds share common characteristics: Top-N values between 6 and 8, chunk sizes of 2,000 tokens, and moderate token budgets (12,000–16,000 tokens). These configurations yield the strongest balance across all metrics, maintaining high levels of answer similarity (approximately 0.901–0.902), well-above-average context recall (0.78–0.79), the highest context precision values in the dataset (0.81–0.83), and stable answer correctness scores. The combination of high precision and competitive recall appears to be the key determinant in maximizing the total score.

The analysis suggests that neither extremely small nor extremely large parameter settings produce optimal RAG performance. Instead, the best results emerge from moderately

high Top-N values (6–8) and mid-sized chunks (around 2,000 tokens), which together create a balanced retrieval environment where the model can access sufficient contextual breadth without sacrificing precision. The highest observed Total score, 2.7589, occurs under the configuration of Top-N = 6 and chunk size = 2,000 (Round 13), indicating that this parameter combination provides the most effective trade-off for the evaluated system.

The best results are achieved at chunk size 2000. It is larger than suggested commonly [34], [5]. However in the study by bhat et al. (2025), the context available for LLM is significantly smaller than in this study. Context recall plays a significant role in this study. The results indicate that increasing the context size leads to improved recall. This can explain the better results with chunk size of 2000 tokens than 500 tokens. The chunk size may vary on the language of the questions and context. In this work, context and answers are in Finnish, which may increase the preferred chunk size. The effect of queried language requires more research in context of rag parameters. The overlap for each chunk size is 25 %. The size of overlap can affect the results and can be studied in the future.

8. CONCLUSIONS

Evaluating the correctness of natural language output is inherently difficult, and accurately identifying the key statements within an answer remains a challenging task without human involvement. In this thesis, we introduced a framework for assessing answer quality using multiple evaluation metrics, enabling a more comprehensive analysis of the overall performance of a RAG model. The optimization method presented here is broadly applicable and demonstrated convergence toward high-performing parameter configurations. It can be extended in future work to explore additional model parameters, and the evaluation score itself can be adjusted to identify the most effective configurations for different applications. In this study, we employed 79 question–ground truth pairs to evaluate each parameter setting; increasing the size of this dataset would likely produce even more robust and generalizable results.

A central difficulty lies in determining the reliability of the evaluation metrics in all contexts. In highly specialized domains, for example, a correct answer may appear incorrect to a general-purpose language model serving as an evaluator. Another notable limitation is the computational cost of the analysis, which is token-intensive. Future work could focus on reducing this cost, allowing for larger datasets and enabling a broader examination of parameter behavior. Due to token constraints, the parameter grid in this thesis was intentionally kept small to test whether the optimization process converges to configurations consistent with prior recommendations. The resulting findings suggest chunk size of approximately 2,000 tokens, which is more than commonly proposed [34], [5]. Although Top-N values have been examined less extensively in previous work, the best-performing configuration identified here was not at either extreme of the tested range. This suggests that the observed optimum may be a local maximum, and further studies with a wider range of Top-N values would provide deeper insight. If the study is replicated, the number of questions, the amount of context available, the type of context, and other RAG parameters can influence the results. However, this study represents results that are a good starting point for future optimizations.

The research questions introduced in Chapter 1 are addressed in this thesis. The relevant evaluation metrics are identified, although their relative weighting may differ depending on the desired outcomes of a RAG model. With respect to the question of optimal parameters, the Results chapter provides well-supported answers, and the best-performing

configurations are presented and discussed. Overall, the results of this chapter indicate that neither extremely small nor extremely large parameter settings lead to optimal RAG performance. Instead, the most effective configurations arise from moderately high Top-N values (approximately 6–8) combined with mid-sized chunk lengths of around 2,000 tokens. These settings provide a balanced retrieval environment, allowing the model to access sufficient contextual breadth while maintaining strong precision. The highest observed Total score, 2.7589, was achieved with Top-N = 6 and a chunk size of 2,000 tokens (Round 13), highlighting this combination as the best-performing trade-off among the evaluated configurations. This outcome reinforces the importance of balancing recall and precision in retrieval design and provides a clear direction for future optimization efforts.

REFERENCES

- [1] Anum Afzal et al. *Towards Optimizing a Retrieval Augmented Generation using Large Language Model on Academic Data*. 2024. arXiv: 2411.08438 [cs.AI]. URL: <https://arxiv.org/abs/2411.08438>.
- [2] Thimira Amaratunga. “Popular LLMs”. In: *Understanding Large Language Models: Learning Their Underlying Concepts and Technologies*. Berkeley, CA: Apress, 2023, pp. 119–130. ISBN: 979-8-8688-0017-7. DOI: 10.1007/979-8-8688-0017-7_5. URL: https://doi.org/10.1007/979-8-8688-0017-7_5.
- [3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2016. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.
- [4] Ashish Bansal. “Optimizing RAG with Hybrid Search and Contextual Chunking”. In: *Journal of Engineering and Applied Sciences Technology* (Aug. 2023), pp. 1–5. DOI: 10.47363/JEAST/2023(5)E114.
- [5] Sinchana Ramakanth Bhat et al. *Rethinking Chunk Size For Long-Document Retrieval: A Multi-Dataset Analysis*. 2025. arXiv: 2505.21700 [cs.IR]. URL: <https://arxiv.org/abs/2505.21700>.
- [6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer, 2006. ISBN: 9780387310732.
- [7] Nikolay Bogoychev. *Not all parameters are born equal: Attention is mostly what you need*. 2021. arXiv: 2010.11859 [cs.CL]. URL: <https://arxiv.org/abs/2010.11859>.
- [8] Denny Britz et al. *Massive Exploration of Neural Machine Translation Architectures*. 2017. arXiv: 1703.03906 [cs.CL]. URL: <https://arxiv.org/abs/1703.03906>.
- [9] Eric Brochu, Vlad M. Cora, and Nando de Freitas. “A Tutorial on Bayesian Optimization of Expensive Cost Functions”. In: *arXiv preprint arXiv:1012.2599* (2010).
- [10] Hongliu Cao. *Recent advances in text embedding: A Comprehensive Review of Top-Performing Methods on the MTEB Benchmark*. 2024. arXiv: 2406.01607 [cs.IR]. URL: <https://arxiv.org/abs/2406.01607>.
- [11] Laura Caspari et al. *Beyond Benchmarks: Evaluating Embedding Model Similarity for Retrieval Augmented Generation Systems*. 2024. arXiv: 2407.08275 [cs.IR]. URL: <https://arxiv.org/abs/2407.08275>.
- [12] Chi-Min Chan et al. *RQ-RAG: Learning to Refine Queries for Retrieval Augmented Generation*. 2024. arXiv: 2404.00610 [cs.CL]. URL: <https://arxiv.org/abs/2404.00610>.

- [13] Nancy Chinchor. "MUC-4 evaluation metrics". In: *Proceedings of the 4th Conference on Message Understanding*. MUC4 '92. McLean, Virginia: Association for Computational Linguistics, 1992, pp. 22–29. ISBN: 1558602739. DOI: 10.3115/1072064.1072067. URL: <https://doi.org/10.3115/1072064.1072067>.
- [14] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. "Reciprocal rank fusion outperforms condorcet and individual rank learning methods". eng. In: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. New York, NY, USA: ACM, 2009, pp. 758–759. ISBN: 1605584835.
- [15] Pádraig Cunningham and Sarah Jane Delany. "k-Nearest Neighbour Classifiers - A Tutorial". In: *ACM Computing Surveys* 54.6 (July 2021), pp. 1–25. ISSN: 1557-7341. DOI: 10.1145/3459665. URL: <http://dx.doi.org/10.1145/3459665>.
- [16] Robert Dale, Hermann Moisl, and H. L. Somers. *Handbook of natural language processing*. eng. 1st ed. New York: Marcel Dekker, 2000. ISBN: 0-429-16449-1.
- [17] Numa. Dhamani. *Introduction to Generative AI*. eng. 1st ed. New York: Manning Publications Co. LLC, 2024. ISBN: 1-63835-434-0.
- [18] Ragas Documentation. *Answer Correctness Metric*. Accessed: 2025-05-06. 2024. URL: https://docs.ragas.io/en/v0.1.21/concepts/metrics/answer_correctness.html.
- [19] Ragas Documentation. *Context Precision*. Accessed: 2024-11-03. 2024. URL: https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/.
- [20] Ragas Documentation. *Context Recall*. Accessed: 2024-11-03. 2024. URL: https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/.
- [21] J. Eisenstein. eng. First edition. The MIT Press, 2019. ISBN: 9780262354578.
- [22] Massimo Esposito et al. *Natural Language Processing: Emerging Neural Approaches and Applications*. eng. Basel: MDPI - Multidisciplinary Digital Publishing Institute, 2022. ISBN: 3-0365-2271-9.
- [23] Duccio Fanelli et al. *A short introduction to Neural Networks and their application to Earth and Materials Science Science*. 2024. arXiv: 2408.11395 [physics.data-an]. URL: <https://arxiv.org/abs/2408.11395>.
- [24] Tom Fawcett. "An introduction to ROC analysis". eng. In: *Pattern recognition letters* 27.8 (2006), pp. 861–874. ISSN: 0167-8655.
- [25] Peter I. Frazier. *A Tutorial on Bayesian Optimization*. 2018. arXiv: 1807.02811 [stat.ML]. URL: <https://arxiv.org/abs/1807.02811>.
- [26] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2024. arXiv: 2312.10997 [cs.CL]. URL: <https://arxiv.org/abs/2312.10997>.
- [27] Roman Garnett. *Bayesian optimization*. eng. Cambridge, United Kingdom: Cambridge University Press, 2023. ISBN: 978-1-108-34897-3.
- [28] T. V. Geetha and S. Sendhilkumar. *Machine Learning : Concepts, Techniques and Applications*. eng. Milton: CRC Press LLC, 2023. ISBN: 9781000867169.

- [29] Exploding Gradients. *Context Precision Metric*. 2025. URL: https://github.com/explodinggradients/ragas/blob/main/src/ragas/metrics/_context_precision.py#L164.
- [30] Donald R. Jones, Matthias Schonlau, and William J. Welch. "Efficient Global Optimization of Expensive Black-Box Functions". eng. In: *Journal of global optimization* 13.4 (1998), pp. 455–492. ISSN: 0925-5001.
- [31] Pranjal Kumar. "Large language models (LLMs): survey, technical frameworks, and future challenges". eng. In: *The Artificial intelligence review* 57.10 (2024), pp. 260–. ISSN: 1573-7462.
- [32] Rangan Majumder et al. *The Science Behind Semantic Search: How AI from Bing is Powering Azure Cognitive Search*. Accessed: 2025-04-28. 2021. URL: <https://www.microsoft.com/en-us/research/blog/the-science-behind-semantic-search-how-ai-from-bing-is-powering-azure-cognitive-search/>.
- [33] Yu A. Malkov and D. A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". eng. In: *IEEE transactions on pattern analysis and machine intelligence* 42.4 (2020), pp. 824–836. ISSN: 0162-8828.
- [34] Microsoft. *Chunk large documents for agentic retrieval and vector search in Azure AI Search*. Accessed: 2025-12-23. 2025. URL: <https://learn.microsoft.com/en-us/azure/search/vector-search-how-to-chunk-documents>.
- [35] Microsoft. *Semantic Ranking - Azure AI Search*. Accessed: 2025-04-28. 2025. URL: <https://learn.microsoft.com/en-us/azure/search/semantic-search-overview>.
- [36] Tomas Mikolov et al. "Efficient Estimation of Word Representations in Vector Space". eng. In: *arXiv.org* (2013). ISSN: 2331-8422.
- [37] Humza Naveed et al. "A Comprehensive Overview of Large Language Models". In: *arXiv preprint arXiv:2307.06435v7* (2023). Version 7. arXiv: 2307.06435v7.
- [38] Minh Nhat Nguyen et al. *Turning Up the Heat: Min-p Sampling for Creative and Coherent LLM Outputs*. 2025. arXiv: 2407.01082 [cs.CL]. URL: <https://arxiv.org/abs/2407.01082>.
- [39] David Louis. Olson and Dursun. Delen. *Advanced data mining techniques*. eng. 1st ed. 2008. Berlin: Springer, 2008. ISBN: 1-281-17968-X.
- [40] OpenAI. *Tokenizer*. Accessed: 2025-12-23. 2025. URL: <https://platform.openai.com/tokenizer>.
- [41] Sarah Packowski et al. *Optimizing and Evaluating Enterprise Retrieval-Augmented Generation (RAG): A Content Design Perspective*. 2024. arXiv: 2410.12812 [cs.IR]. URL: <https://arxiv.org/abs/2410.12812>.
- [42] Ioannis Papadimitriou et al. *RAG Playground: A Framework for Systematic Evaluation of Retrieval Strategies and Prompt Engineering in RAG Systems*. 2024. arXiv: 2412.12322 [cs.LG]. URL: <https://arxiv.org/abs/2412.12322>.

- [43] Bo Peng et al. *RWKV: Reinventing RNNs for the Transformer Era*. 2023. arXiv: 2305.13048 [cs.CL]. URL: <https://arxiv.org/abs/2305.13048>.
- [44] Jeffrey Pennington, Richard Socher, and Christopher Manning. "Glove: Global Vectors for Word Representation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Vol. 14. Jan. 2014, pp. 1532–1543. DOI: 10.3115/v1/D14-1162.
- [45] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. Technical report. OpenAI, 2019. URL: <https://openai.com/index/language-unsupervised/>.
- [46] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. *Searching for Activation Functions*. 2017. arXiv: 1710.05941 [cs.NE]. URL: <https://arxiv.org/abs/1710.05941>.
- [47] Carl Edward. Rasmussen and Christopher K. I. Williams. *Gaussian processes for machine learning*. eng. 1st ed. Adaptive computation and machine learning. Cambridge, Mass: MIT Press, 2006. ISBN: 0-262-26107-3.
- [48] S. E Robertson and Hugo Zaragoza. "The probabilistic relevance framework: BM25 and beyond". eng. In: *Foundations and trends in information retrieval* 3.4 (2009), pp. 333–389. ISSN: 1554-0669.
- [49] Denis Rothman. *RAG-Driven Generative AI: Build custom retrieval augmented generation pipelines with LlamaIndex, Deep Lake, and Pinecone*. eng. 1st ed. Birmingham: Packt Publishing, 2024. ISBN: 9781836200918.
- [50] Tolga Şakar and Hakan Emekci. "Maximizing RAG efficiency: A comparative analysis of RAG methods". In: *Natural Language Processing* 31 (Oct. 2024), pp. 1–25. DOI: 10.1017/nlp.2024.53.
- [51] Claude. Sammut and Geoffrey I. Webb. *Encyclopedia of Machine Learning*. eng. 1st ed. 2010. New York, NY: Springer US, 2010. ISBN: 1-280-38505-7.
- [52] Yutaka Sasaki. "The truth of the F-measure". In: *Teach Tutor Mater* (Jan. 2007).
- [53] Kunal Sawarkar, Abhilasha Mangal, and Shivam Raj Solanki. "Blended RAG: Improving RAG (Retriever-Augmented Generation) Accuracy with Semantic Search and Hybrid Query-Based Retrievers". In: *2024 IEEE 7th International Conference on Multimedia Information Processing and Retrieval (MIPR)*. Vol. 24. IEEE, Aug. 2024, pp. 155–161. DOI: 10.1109/mipr62202.2024.00031. URL: <http://dx.doi.org/10.1109/MIPR62202.2024.00031>.
- [54] Robin M. Schmidt. *Recurrent Neural Networks (RNNs): A gentle Introduction and Overview*. 2019. arXiv: 1912.05911 [cs.LG]. URL: <https://arxiv.org/abs/1912.05911>.
- [55] Spurthi Setty et al. *Improving Retrieval for RAG based Question Answering Models on Financial Documents*. 2024. arXiv: 2404.07221 [cs.IR]. URL: <https://arxiv.org/abs/2404.07221>.
- [56] Bobak Shahriari et al. "Taking the Human Out of the Loop: A Review of Bayesian Optimization". In: *Proceedings of the IEEE* 104.1 (2016), pp. 148–175.

- [57] Shivam R. Solanki and Drupad K. Khublani. *Generative Artificial Intelligence : Exploring the Power and Potential of Generative AI*. eng. 1st ed. 2024. Berkeley, CA: Apress, 2024. ISBN: 9798868804038.
- [58] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [59] Xiaohua Wang et al. *Searching for Best Practices in Retrieval-Augmented Generation*. 2024. arXiv: 2407.01219 [cs.CL]. URL: <https://arxiv.org/abs/2407.01219>.
- [60] Zhiruo Wang et al. *Learning to Filter Context for Retrieval-Augmented Generation*. 2023. arXiv: 2311.08377 [cs.CL]. URL: <https://arxiv.org/abs/2311.08377>.
- [61] Sophie Xhonneux et al. *In-Context Learning Can Re-learn Forbidden Tasks*. 2024. arXiv: 2402.05723 [cs.LG]. URL: <https://arxiv.org/abs/2402.05723>.
- [62] Shi-Qi Yan et al. *Corrective Retrieval Augmented Generation*. 2024. arXiv: 2401.15884 [cs.CL]. URL: <https://arxiv.org/abs/2401.15884>.
- [63] Zijie Zhong et al. *Mix-of-Granularity: Optimize the Chunking Granularity for Retrieval-Augmented Generation*. 2025. arXiv: 2406.00456 [cs.LG]. URL: <https://arxiv.org/abs/2406.00456>.
- [64] Yuqi Zhu et al. *Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models*. 2023. arXiv: 2309.02772 [cs.SE]. URL: <https://arxiv.org/abs/2309.02772>.

APPENDIX A: RAG CONTEXT DOCUMENTS

Table A.1. *RAG Context Documents. The documents, questions and answers in this thesis are in Finnish.*

Name	Token Count	File Type
Government Proposal to Parliament on the Occupational Accidents and Diseases Act and Related Laws	425,059	pdf
Occupational Accident and Disease, Kirsi Salo	185,983	pdf
Insurance Handbook	155,432	pdf
Occupational Accidents and Diseases Condition Survey (TYTAL)	102,105	pdf
Occupational Accident and Disease Insurance Guidebook	85,668	docs
Occupational Accidents and Diseases Act	76,120	pdf
Occupational Diseases Handling Guide	44,150	pdf
Accident Insurance Center Training 23.4.2021	39,796	ppt
Compensable Conditions in Occupational Accident and Disease Cases	25,867	docs
Paper Union – Occupational Accident and Disease Guide	12,985	pdf
Accident Compensation Board Info 28.11.2024	5,449	ppt
Accident Compensation Board Info 16.2.2024	5,280	ppt
Accident Compensation Board Info 6.2.2025	4,519	ppt

APPENDIX B: TEST SETUP PARAMETERS

Table B.1. Configuration Parameters

Parameter	Value
Embedding Model	text-embedding-ada-002
LLM	GPT-4o
Document Store	Blob Storage
Retrieval	Hybrid Search + Semantic Reranker
Vector Search Algorithm	HNSW
efConstruction	400
efSearch	500
Similarity Metric	Cosine
Max Tokens	800
Temperature	0.0
Top P	0.95
Frequency Penalty	0
Presence Penalty	0
Filter	None
Strictness	3
Top-N	Optimized
Chunk Size	Optimized